

## LAB # 1

**Objective:** Use PostgreSQL to master environment configuration and analytical SQL operations.

### 1.3. EXERCISES

For all tasks, use the Snipping Tool to take focused screenshots. Your screenshots should be clear and readable.

#### 1.3.1 Task 01

1. Create a table: *library\_logs* with columns *book\_title*, *genre*, *borrow\_duration\_days*, *fine\_amount*, and *return\_status*. The *return\_status* column can have values Returned, Overdue, or NULL.

> **Input**

```
CREATE TABLE library_logs (  
    book_title VARCHAR(255),  
    genre VARCHAR(50),  
    borrow_duration_days INT,  
    fine_amount DECIMAL(5,2),  
    return_status VARCHAR(10)  
);
```

 **Library\_logs [-]**

- book\_title [varchar(255)]
- genre [varchar(50)]
- borrow\_duration\_days [int]
- fine\_amount [decimal(5,2)]
- return\_status [varchar(10)]

Library\_logs

| book_title | genre | borrow_duration_days | fine_amount | return_status |
|------------|-------|----------------------|-------------|---------------|
| empty      |       |                      |             |               |

2. Insert 10 rows. Ensure at least 3 rows have a *NULL* *return\_status* and 4 different genres.

>
Input

Run SQL

```

INSERT INTO library_logs (book_title, genre, borrow_duration_days, fine_amount, return_status) VALUES
('The Great Gatsby', 'Fiction', 14, 0.50, 'Returned'),
('A Brief History of Time', 'Science', 21, 1.25, 'Overdue'),
('The Art of Data Analysis', 'Technology', 10, NULL, NULL),
('Database Systems', 'Technology', 7, 0.00, 'Returned'),
('The Second World War', 'History', 30, 2.00, 'Overdue'),
('Data Science for Beginners', 'Technology', 12, NULL, NULL),
('Introduction to Algorithms', 'Technology', 21, 0.75, 'Returned'),
('The Silent Patient', 'Fiction', 18, NULL, NULL),
('Sapiens: A Brief History of Humankind', 'History', 28, 1.50, 'Returned'),
('System Design Interview', 'Technology', 15, 0.25, 'Returned');

```

Library\_logs

| book_title                            | genre      | borrow_duration_days | fine_amount | return_status |
|---------------------------------------|------------|----------------------|-------------|---------------|
| The Great Gatsby                      | Fiction    | 14                   | 0.5         | Returned      |
| A Brief History of Time               | Science    | 21                   | 1.25        | Overdue       |
| The Art of Data Analysis              | Technology | 10                   |             |               |
| Database Systems                      | Technology | 7                    | 0           | Returned      |
| The Second World War                  | History    | 30                   | 2           | Overdue       |
| Data Science for Beginners            | Technology | 12                   |             |               |
| Introduction to Algorithms            | Technology | 21                   | 0.75        | Returned      |
| The Silent Patient                    | Fiction    | 18                   |             |               |
| Sapiens: A Brief History of Humankind | History    | 28                   | 1.5         | Returned      |
| System Design Interview               | Technology | 15                   | 0.25        | Returned      |

3. Find the average borrow duration for each genre.

>
Input

```

SELECT genre, AVG(borrow_duration_days) AS avg_borrow_duration
FROM library_logs
GROUP BY genre;

```

Output

| genre      | avg_borrow_duration |
|------------|---------------------|
| Fiction    | 16                  |
| History    | 29                  |
| Science    | 21                  |
| Technology | 13                  |

4. Show all fines, replacing *NULL* with 0.00.

## &gt; Input

```
SELECT book_title, genre, borrow_duration_days,
       COALESCE(fine_amount, 0.00) AS fine_amount,
       return_status
FROM library_logs;
```

## Output

| book_title                            | genre      | borrow_duration_days | fine_amount | return_status |
|---------------------------------------|------------|----------------------|-------------|---------------|
| The Great Gatsby                      | Fiction    | 14                   | 0.5         | Returned      |
| A Brief History of Time               | Science    | 21                   | 1.25        | Overdue       |
| The Art of Data Analysis              | Technology | 10                   | 0           |               |
| Database Systems                      | Technology | 7                    | 0           | Returned      |
| The Second World War                  | History    | 30                   | 2           | Overdue       |
| Data Science for Beginners            | Technology | 12                   | 0           |               |
| Introduction to Algorithms            | Technology | 21                   | 0.75        | Returned      |
| The Silent Patient                    | Fiction    | 18                   | 0           |               |
| Sapiens: A Brief History of Humankind | History    | 28                   | 1.5         | Returned      |
| System Design Interview               | Technology | 15                   | 0.25        | Returned      |

5. List only the genres that have more than 2 books on your list.

## &gt; Input

```
SELECT genre, COUNT(*) AS book_count
FROM library_logs
GROUP BY genre
HAVING COUNT(*) > 2;
```

## Output

| genre      | book_count |
|------------|------------|
| Technology | 5          |

6. Find all books with the words “Data” or “System” in the title.

## &gt; Input

```
SELECT *
FROM library_logs
WHERE book_title LIKE '%Data%' OR book_title LIKE '%System%';
```

## Output

| book_title                 | genre      | borrow_duration_days | fine_amount | return_status |
|----------------------------|------------|----------------------|-------------|---------------|
| The Art of Data Analysis   | Technology | 10                   |             |               |
| Database Systems           | Technology | 7                    | 0           | Returned      |
| Data Science for Beginners | Technology | 12                   |             |               |
| System Design Interview    | Technology | 15                   | 0.25        | Returned      |

## 1.3.2. Task 02

1. Create a table *borrowers* with columns *borrower\_id* (PK), *borrower\_name*, *membership\_tire* (Gold, Silver, Bronze), *joined\_date*, and *book\_title*. The *book\_title* column lists the book the borrower has borrowed.

> **Input**

```
CREATE TABLE borrowers (
  borrower_id INTEGER PRIMARY KEY,
  borrower_name TEXT,
  membership_tier TEXT CHECK (membership_tier IN ('Gold', 'Silver', 'Bronze')),
  joined_date TEXT, -- SQLite uses TEXT for dates; other DBMS can use DATE
  book_title TEXT
);
```

 **Borrowers [-]**

- borrower\_id [integer]
- borrower\_name [text]
- membership\_tier [text]
- joined\_date [text]
- book\_title [text]

#### Borrowers

| borrower_id | borrower_name | membership_tier | joined_date | book_title |
|-------------|---------------|-----------------|-------------|------------|
| empty       |               |                 |             |            |

2. Insert 8 rows into the *borrowers* table. Ensure that some borrowers have *book\_title* values from TASK 1. Also, ensure that at least 2 borrowers have *NULL* values in *book\_title*.

> **Input**

```
INSERT INTO borrowers (borrower_id, borrower_name, membership_tier, joined_date, book_title) VALUES
(1, 'Alice Johnson', 'Gold', '2023-01-15', 'The Great Gatsby'),
(2, 'Bob Smith', 'Silver', '2023-03-20', 'Database Systems'),
(3, 'Carol White', 'Bronze', '2023-06-10', 'The Art of Data Analysis'),
(4, 'David Brown', 'Gold', '2023-02-05', NULL),
(5, 'Eve Davis', 'Silver', '2023-07-12', 'Sapiens: A Brief History of Humankind'),
(6, 'Frank Miller', 'Bronze', '2023-04-18', 'System Design Interview'),
(7, 'Grace Wilson', 'Gold', '2023-05-22', NULL),
(8, 'Henry Taylor', 'Silver', '2023-08-30', 'Data Science for Beginners');
```

**Borrowers**

| borrower_id | borrower_name | membership_tier | joined_date | book_title                            |
|-------------|---------------|-----------------|-------------|---------------------------------------|
| 1           | Alice Johnson | Gold            | 2023-01-15  | The Great Gatsby                      |
| 2           | Bob Smith     | Silver          | 2023-03-20  | Database Systems                      |
| 3           | Carol White   | Bronze          | 2023-06-10  | The Art of Data Analysis              |
| 4           | David Brown   | Gold            | 2023-02-05  |                                       |
| 5           | Eve Davis     | Silver          | 2023-07-12  | Sapiens: A Brief History of Humankind |
| 6           | Frank Miller  | Bronze          | 2023-04-18  | System Design Interview               |
| 7           | Grace Wilson  | Gold            | 2023-05-22  |                                       |
| 8           | Henry Taylor  | Silver          | 2023-08-30  | Data Science for Beginners            |

3. List the borrowers and their corresponding fines for all the books that have been logged.

**Input**

```
SELECT b.borrower_id, b.borrower_name, b.membership_tier,
       l.book_title, l.fine_amount
FROM borrowers b
INNER JOIN library_logs l ON b.book_title = l.book_title;
```

**Output**

| borrower_id | borrower_name | membership_tier | book_title                            | fine_amount |
|-------------|---------------|-----------------|---------------------------------------|-------------|
| 1           | Alice Johnson | Gold            | The Great Gatsby                      | 0.5         |
| 2           | Bob Smith     | Silver          | Database Systems                      | 0           |
| 3           | Carol White   | Bronze          | The Art of Data Analysis              |             |
| 5           | Eve Davis     | Silver          | Sapiens: A Brief History of Humankind | 1.5         |
| 6           | Frank Miller  | Bronze          | System Design Interview               | 0.25        |
| 8           | Henry Taylor  | Silver          | Data Science for Beginners            |             |

4. Show all the borrowers and their book return status. Ensure borrowers without a log entry show *NULL* for the status.

**Input**

```
SELECT b.borrower_id, b.borrower_name, b.membership_tier,
       b.book_title, l.return_status
FROM borrowers b
LEFT JOIN library_logs l ON b.book_title = l.book_title;
```

## Output

| borrower_id | borrower_name | membership_tier | book_title                            | return_status |
|-------------|---------------|-----------------|---------------------------------------|---------------|
| 1           | Alice Johnson | Gold            | The Great Gatsby                      | Returned      |
| 2           | Bob Smith     | Silver          | Database Systems                      | Returned      |
| 3           | Carol White   | Bronze          | The Art of Data Analysis              |               |
| 4           | David Brown   | Gold            |                                       |               |
| 5           | Eve Davis     | Silver          | Sapiens: A Brief History of Humankind | Returned      |
| 6           | Frank Miller  | Bronze          | System Design Interview               | Returned      |
| 7           | Grace Wilson  | Gold            |                                       |               |
| 8           | Henry Taylor  | Silver          | Data Science for Beginners            |               |

5. Find the total fine accumulated by each membership tier.

## &gt; Input

```
SELECT b.membership_tier, COALESCE(SUM(l.fine_amount), 0) AS total_fine
FROM borrowers b
LEFT JOIN library_logs l ON b.book_title = l.book_title
GROUP BY b.membership_tier;
```

## Output

| membership_tier | total_fine |
|-----------------|------------|
| Bronze          | 0.25       |
| Gold            | 0.5        |
| Silver          | 1.5        |

6. List only the membership tier groups where the average fine is greater than \$5.00.

## &gt; Input

```
SELECT b.membership_tier, AVG(l.fine_amount) AS avg_fine
FROM borrowers b
INNER JOIN library_logs l ON b.book_title = l.book_title
GROUP BY b.membership_tier
HAVING AVG(l.fine_amount) > 5.00;
```

## Output

SQL query successfully executed. However, the result set is empty.

## LAB # 2

**Objective:** Use data profiling techniques and programmatic cleaning scripts to clean data.

### 2.3. EXERCISES

For all tasks, use the Snipping Tool to take focused screenshots. Your screenshots should be clear and readable.

#### 2.3.1 Task 01

1. Create a table *patients* with columns *patient\_id*, *patient\_name*, *blood\_group*, *last\_visit\_date*, and *emergency\_contact*.

<

Input

```
CREATE TABLE patients (
  patient_id INT PRIMARY KEY,
  patient_name VARCHAR(100),
  blood_group VARCHAR(20),
  last_visit_date DATE,
  emergency_contact VARCHAR(50)
);
```

**Patients [-]**

|                                 |
|---------------------------------|
| patient_id [int]                |
| patient_name [varchar(100)]     |
| blood_group [varchar(20)]       |
| last_visit_date [date]          |
| emergency_contact [varchar(50)] |

**Patients**

| patient_id | patient_name | blood_group | last_visit_date | emergency_contact |
|------------|--------------|-------------|-----------------|-------------------|
| empty      |              |             |                 |                   |

2. Insert 10 rows. Ensure that:

- a. At least 2 rows are exact duplicates
- b. There is inconsistency in blood group e.g., 'A+', 'a positive', 'A plus', etc.
- c. There are missing (*NULL*) emergency contacts.
- d. There are names with accidental symbols such as 'John%' or digits such as 'John123'.

```

Input
INSERT INTO patients (patient_id, patient_name, blood_group, last_visit_date, emergency_contact)
VALUES
(1, 'John Smith', 'A+', '2024-01-10', '555-1234'),
(2, 'Jane Doe', 'a positive', '2024-01-12', NULL),
(3, 'Bob%Jones', 'B+', '2024-02-01', '555-5678'),
(4, 'Alice123', 'O-', '2024-02-03', '555-8765'),
(5, 'John Smith', 'A+', '2024-01-10', '555-1234'), -- exact duplicate of row 1
(6, 'Charlie Brown', 'AB plus', '2024-02-10', NULL),
(7, 'Diana@Prince', 'A+', '2024-02-15', '555-4321'),
(8, 'Eve#Davis', 'b positive', '2024-02-20', '555-9999'),
(9, 'Frank', 'O+', '2024-02-22', NULL),
(10, 'Grace Hopper', 'A+', '2024-02-25', '555-1111');

```

Patients

| patient_id | patient_name  | blood_group | last_visit_date | emergency_contact |
|------------|---------------|-------------|-----------------|-------------------|
| 1          | John Smith    | A+          | 2024-01-10      | 555-1234          |
| 2          | Jane Doe      | a positive  | 2024-01-12      |                   |
| 3          | Bob%Jones     | B+          | 2024-02-01      | 555-5678          |
| 4          | Alice123      | O-          | 2024-02-03      | 555-8765          |
| 5          | John Smith    | A+          | 2024-01-10      | 555-1234          |
| 6          | Charlie Brown | AB plus     | 2024-02-10      |                   |
| 7          | Diana@Prince  | A+          | 2024-02-15      | 555-4321          |
| 8          | Eve#Davis     | b positive  | 2024-02-20      | 555-9999          |
| 9          | Frank         | O+          | 2024-02-22      |                   |
| 10         | Grace Hopper  | A+          | 2024-02-25      | 555-1111          |

3. Write a SQL script to:

- a. a. Count how many patients are missing emergency contacts.

```

Input
SELECT COUNT(*) AS missing_emergency_contact_count
FROM patients
WHERE emergency_contact IS NULL OR emergency_contact = '';

Output
missing_emergency_contact_count
3

```

- b. b. Standardize all blood groups to be Uppercase, e.g., 'A+'.

```

Input
UPDATE patients
SET blood_group =
CASE
    WHEN UPPER(TRIM(blood_group)) IN ('A+', 'A POSITIVE', 'A PLUS') THEN 'A+'
    WHEN UPPER(TRIM(blood_group)) IN ('A-', 'A NEGATIVE', 'A MINUS') THEN 'A-'
    WHEN UPPER(TRIM(blood_group)) IN ('B+', 'B POSITIVE', 'B PLUS') THEN 'B+'
    WHEN UPPER(TRIM(blood_group)) IN ('B-', 'B NEGATIVE', 'B MINUS') THEN 'B-'
    WHEN UPPER(TRIM(blood_group)) IN ('O+', 'O POSITIVE', 'O PLUS') THEN 'O+'
    WHEN UPPER(TRIM(blood_group)) IN ('O-', 'O NEGATIVE', 'O MINUS') THEN 'O-'
    WHEN UPPER(TRIM(blood_group)) IN ('AB+', 'AB POSITIVE', 'AB PLUS') THEN 'AB+'
    WHEN UPPER(TRIM(blood_group)) IN ('AB-', 'AB NEGATIVE', 'AB MINUS') THEN 'AB-'
    ELSE UPPER(TRIM(blood_group)) -- fallback: just uppercase and trimmed
END;

```

Patients

| patient_id | patient_name  | blood_group | last_visit_date | emergency_contact |
|------------|---------------|-------------|-----------------|-------------------|
| 1          | John Smith    | A+          | 2024-01-10      | 555-1234          |
| 2          | Jane Doe      | A+          | 2024-01-12      |                   |
| 3          | Bob%Jones     | B+          | 2024-02-01      | 555-5678          |
| 4          | Alice123      | O-          | 2024-02-03      | 555-8765          |
| 5          | John Smith    | A+          | 2024-01-10      | 555-1234          |
| 6          | Charlie Brown | AB+         | 2024-02-10      |                   |
| 7          | Diana@Prince  | A+          | 2024-02-15      | 555-4321          |
| 8          | Eve#Davis     | B+          | 2024-02-20      | 555-9999          |
| 9          | Frank         | O+          | 2024-02-22      |                   |
| 10         | Grace Hopper  | A+          | 2024-02-25      | 555-1111          |

c. c. Clean the names by removing leading and trailing spaces.

#### < Input

```
-- Remove leading/trailing spaces
UPDATE patients
SET patient_name = TRIM(patient_name);

-- Remove digits 0-9
UPDATE patients SET patient_name = REPLACE(patient_name, '0', '');
UPDATE patients SET patient_name = REPLACE(patient_name, '1', '');
UPDATE patients SET patient_name = REPLACE(patient_name, '2', '');
UPDATE patients SET patient_name = REPLACE(patient_name, '3', '');
UPDATE patients SET patient_name = REPLACE(patient_name, '4', '');
UPDATE patients SET patient_name = REPLACE(patient_name, '5', '');
UPDATE patients SET patient_name = REPLACE(patient_name, '6', '');
UPDATE patients SET patient_name = REPLACE(patient_name, '7', '');
UPDATE patients SET patient_name = REPLACE(patient_name, '8', '');
UPDATE patients SET patient_name = REPLACE(patient_name, '9', '');

-- Remove specific symbols from the sample data
UPDATE patients SET patient_name = REPLACE(patient_name, '%', '');
UPDATE patients SET patient_name = REPLACE(patient_name, '@', '');
UPDATE patients SET patient_name = REPLACE(patient_name, '#', '');

UPDATE patients SET patient_name = TRIM(patient_name);
```

Patients

| patient_id | patient_name  | blood_group | last_visit_date | emergency_contact |
|------------|---------------|-------------|-----------------|-------------------|
| 1          | John Smith    | A+          | 2024-01-10      | 555-1234          |
| 2          | Jane Doe      | A+          | 2024-01-12      |                   |
| 3          | BobJones      | B+          | 2024-02-01      | 555-5678          |
| 4          | Alice         | O-          | 2024-02-03      | 555-8765          |
| 5          | John Smith    | A+          | 2024-01-10      | 555-1234          |
| 6          | Charlie Brown | AB+         | 2024-02-10      |                   |
| 7          | DianaPrince   | A+          | 2024-02-15      | 555-4321          |
| 8          | EveDavis      | B+          | 2024-02-20      | 555-9999          |
| 9          | Frank         | O+          | 2024-02-22      |                   |
| 10         | Grace Hopper  | A+          | 2024-02-25      | 555-1111          |

- d. Remove any numbers or special characters from patient names.

| Input                                                                                                             |               |
|-------------------------------------------------------------------------------------------------------------------|---------------|
| <pre>SELECT blood_group, COUNT(*) AS patient_count FROM patients GROUP BY blood_group ORDER BY blood_group;</pre> |               |
| Output                                                                                                            |               |
| blood_group                                                                                                       | patient_count |
| A+                                                                                                                | 5             |
| AB+                                                                                                               | 1             |
| B+                                                                                                                | 2             |
| O+                                                                                                                | 1             |
| O-                                                                                                                | 1             |

4. Show a list of unique blood groups and how many patients belong to each.

| Input                                                                                                             |               |
|-------------------------------------------------------------------------------------------------------------------|---------------|
| <pre>SELECT blood_group, COUNT(*) AS patient_count FROM patients GROUP BY blood_group ORDER BY blood_group;</pre> |               |
| Output                                                                                                            |               |
| blood_group                                                                                                       | patient_count |
| A+                                                                                                                | 5             |
| AB+                                                                                                               | 1             |
| B+                                                                                                                | 2             |
| O+                                                                                                                | 1             |
| O-                                                                                                                | 1             |

## **LAB # 3**

**Objective:** Use Materialized Views to transform OLTPs into de-normalized data warehouses

### **3.3. EXERCISES**

*For all tasks, use the Snipping Tool to take focused screenshots. Your screenshots should be clear and readable.*

#### **3.3.1. Task 01**

1. Create a new database *university\_dw*.
2. Create four tables.
  - Courses (Course ID, Course Name, Credits).
  - Instructors (Instructor ID, Instructor Name, Department ID)
  - Departments (Department ID, Department Name)
  - Enrollments (Enrollment ID, Student Name, Course ID, Instructor ID)
3. Populate the tables.
  - a. Insert 5 courses.
  - b. Insert 5 instructors.
  - c. Insert 3 departments.
  - d. Insert 10 enrollment records.
4. Create a Materialized View, *mv\_enrollment\_flat*, that shows the student's name, course name, credits, instructor name, and department name.
5. The University policy is to allocate the budget based on the actual teaching load of a department. Rank the departments in order of total credits.
6. The University wants to determine if any faculty member is "overloaded". Identify the top three faculty members and their departments that handle the most students.
7. The University wants to allocate classrooms based on the number of students enrolled in a course. For each course, list the unique number of students enrolled and the department overseeing it.
8. Add a new student enrollment to the Enrollments table. Then, display the contents of the Materialized View. Does the new student appear? Explain your result.
9. Run the following command:  

```
REFRESH MATERIALIZED VIEW mv_enrollment_flat;
```

  
Display the contents of the Materialized View again. Is there any change? Explain.
10. In a real-world Big Data environment, explain why we might choose to Refresh a view once a night (at 2 AM) rather than every time a new row is added.

**Source Code:****00\_create\_database.sql:**

| Query | Query History                     |
|-------|-----------------------------------|
| 1     | -- Khalid Hussain (2022F-BCE-130) |
| 2     |                                   |
| 3     | -- Task 1: Create the database.   |
| 4     | CREATE DATABASE university_dw;    |
| 5     |                                   |
| 6     | -- Khalid Hussain (2022F-BCE-130) |

---

Data Output   **Messages**   Notifications

ERROR: database "university\_dw" already exists

SQL state: 42P04

**01\_schema\_and\_seed.sql:**

| Query | Query History                                                                          |
|-------|----------------------------------------------------------------------------------------|
| 1     | -- Khalid Hussain (2022F-BCE-130)                                                      |
| 2     |                                                                                        |
| 3     | -- Task 2: Create the tables.                                                          |
| 4     | DROP MATERIALIZED VIEW IF EXISTS mv_enrollment_flat;                                   |
| 5     | DROP TABLE IF EXISTS enrollments;                                                      |
| 6     | DROP TABLE IF EXISTS instructors;                                                      |
| 7     | DROP TABLE IF EXISTS courses;                                                          |
| 8     | DROP TABLE IF EXISTS departments;                                                      |
| 9     |                                                                                        |
| 10    | CREATE TABLE departments (                                                             |
| 11    | department_id INT PRIMARY KEY,                                                         |
| 12    | department_name VARCHAR(100) NOT NULL                                                  |
| 13    | );                                                                                     |
| 14    |                                                                                        |
| 15    | CREATE TABLE courses (                                                                 |
| 16    | course_id INT PRIMARY KEY,                                                             |
| 17    | course_name VARCHAR(100) NOT NULL,                                                     |
| 18    | credits INT NOT NULL CHECK (credits > 0)                                               |
| 19    | );                                                                                     |
| 20    |                                                                                        |
| 21    | CREATE TABLE instructors (                                                             |
| 22    | instructor_id INT PRIMARY KEY,                                                         |
| 23    | instructor_name VARCHAR(100) NOT NULL,                                                 |
| 24    | department_id INT NOT NULL REFERENCES departments(department_id)                       |
| 25    | );                                                                                     |
| 26    |                                                                                        |
| 27    | CREATE TABLE enrollments (                                                             |
| 28    | enrollment_id INT PRIMARY KEY,                                                         |
| 29    | student_name VARCHAR(100) NOT NULL,                                                    |
| 30    | course_id INT NOT NULL REFERENCES courses(course_id),                                  |
| 31    | instructor_id INT NOT NULL REFERENCES instructors(instructor_id)                       |
| 32    | );                                                                                     |
| 33    |                                                                                        |
| 34    | -- Task 3: Populate the tables.                                                        |
| 35    | INSERT INTO departments (department_id, department_name) VALUES                        |
| 36    | (1, 'Computer Science'),                                                               |
| 37    | (2, 'Data Science'),                                                                   |
| 38    | (3, 'Electrical Engineering');                                                         |
| 39    |                                                                                        |
| 40    | INSERT INTO courses (course_id, course_name, credits) VALUES                           |
| 41    | (101, 'Database Systems', 3),                                                          |
| 42    | (102, 'Data Warehousing', 4),                                                          |
| 43    | (103, 'Machine Learning', 4),                                                          |
| 44    | (104, 'Big Data Analytics', 3),                                                        |
| 45    | (105, 'Digital Logic Design', 3);                                                      |
| 46    |                                                                                        |
| 47    | INSERT INTO instructors (instructor_id, instructor_name, department_id) VALUES         |
| 48    | (1, 'Miss Arifa', 1),                                                                  |
| 49    | (2, 'Teacher 2', 1),                                                                   |
| 50    | (3, 'Teacher 3', 2),                                                                   |
| 51    | (4, 'Teacher 4', 2),                                                                   |
| 52    | (5, 'Teacher 5', 3);                                                                   |
| 53    |                                                                                        |
| 54    | INSERT INTO enrollments (enrollment_id, student_name, course_id, instructor_id) VALUES |
| 55    | (1, 'Sheikh Khalid', 101, 1),                                                          |
| 56    | (2, 'Khalid Hussain', 101, 1),                                                         |
| 57    | (3, 'Sheikh Khalid Hussain', 102, 3),                                                  |
| 58    | (4, 'Student 4', 102, 3),                                                              |
| 59    | (5, 'Student 5', 103, 4),                                                              |
| 60    | (6, 'Student 6', 103, 4),                                                              |

```

61      (7, 'Student 7', 104, 2),
62      (8, 'Student 8', 105, 5),
63      (9, 'Student 9', 102, 3),
64      (10, 'Student 10', 102, 3);
65
66 SELECT 'Departments' AS table_name, COUNT(*) AS total_rows FROM departments
67 UNION ALL
68 SELECT 'Courses', COUNT(*) FROM courses
69 UNION ALL
70 SELECT 'Instructors', COUNT(*) FROM instructors
71 UNION ALL
72 SELECT 'Enrollments', COUNT(*) FROM enrollments;
73
74 -- Khalid Hussain (2022F-BCE-130)

```

**02\_materialized\_view\_and\_queries.sql:**

| Query | Query History                                                                          |
|-------|----------------------------------------------------------------------------------------|
| 1     | -- Khalid Hussain (2022F-BCE-130)                                                      |
| 2     |                                                                                        |
| 3     | -- Task 4: Create the materialized view.                                               |
| 4     | DROP MATERIALIZED VIEW IF EXISTS mv_enrollment_flat;                                   |
| 5     | CREATE MATERIALIZED VIEW mv_enrollment_flat AS                                         |
| 6     | SELECT                                                                                 |
| 7     | e.student_name,                                                                        |
| 8     | c.course_name,                                                                         |
| 9     | c.credits,                                                                             |
| 10    | i.instructor_name,                                                                     |
| 11    | d.department_name                                                                      |
| 12    | FROM enrollments e                                                                     |
| 13    | JOIN courses c                                                                         |
| 14    | ON e.course_id = c.course_id                                                           |
| 15    | JOIN instructors i                                                                     |
| 16    | ON e.instructor_id = i.instructor_id                                                   |
| 17    | JOIN departments d                                                                     |
| 18    | ON i.department_id = d.department_id;                                                  |
| 19    |                                                                                        |
| 20    | SELECT *                                                                               |
| 21    | FROM mv_enrollment_flat                                                                |
| 22    | ORDER BY department_name, instructor_name, course_name, student_name;                  |
| 23    |                                                                                        |
| 24    | -- Task 5: Rank departments by total credits handled.                                  |
| 25    | SELECT                                                                                 |
| 26    | DENSE_RANK() OVER (ORDER BY SUM(credits) DESC) AS department_rank,                     |
| 27    | department_name,                                                                       |
| 28    | SUM(credits) AS total_credits                                                          |
| 29    | FROM mv_enrollment_flat                                                                |
| 30    | GROUP BY department_name                                                               |
| 31    | ORDER BY department_rank, department_name;                                             |
| 32    |                                                                                        |
| 33    | -- Task 6: Identify the top three faculty members handling the most students.          |
| 34    | SELECT                                                                                 |
| 35    | instructor_name,                                                                       |
| 36    | department_name,                                                                       |
| 37    | COUNT(DISTINCT student_name) AS students_handled                                       |
| 38    | FROM mv_enrollment_flat                                                                |
| 39    | GROUP BY instructor_name, department_name                                              |
| 40    | ORDER BY students_handled DESC, instructor_name                                        |
| 41    | LIMIT 3;                                                                               |
| 42    |                                                                                        |
| 43    | -- Task 7: List unique students enrolled in each course and the overseeing department. |
| 44    | SELECT                                                                                 |
| 45    | course_name,                                                                           |
| 46    | department_name,                                                                       |
| 47    | COUNT(DISTINCT student_name) AS unique_students_enrolled                               |
| 48    | FROM mv_enrollment_flat                                                                |
| 49    | GROUP BY course_name, department_name                                                  |
| 50    | ORDER BY course_name, department_name;                                                 |
| 51    |                                                                                        |
| 52    | -- Khalid Hussain (2022F-BCE-130)                                                      |

## 03\_refresh\_demo.sql:

```

Query  Query History
1  -- Khalid Hussain (2022F-BCE-130)
2
3  DELETE FROM enrollments
4  WHERE enrollment_id = 11;
5
6  REFRESH MATERIALIZED VIEW mv_enrollment_flat;
7
8  -- Task 8: Add a new enrollment and display the materialized view before refresh.
9  INSERT INTO enrollments (enrollment_id, student_name, course_id, instructor_id)
10 VALUES (11, 'Student 11', 104, 2);
11
12 SELECT *
13 FROM enrollments
14 WHERE enrollment_id = 11;
15
16 SELECT *
17 FROM mv_enrollment_flat
18 WHERE student_name = 'Student 11';
19
20 SELECT *
21 FROM mv_enrollment_flat
22 ORDER BY department_name, instructor_name, course_name, student_name;
23
24 -- Task 9: Refresh the materialized view and display it again.
25 REFRESH MATERIALIZED VIEW mv_enrollment_flat;
26
27 SELECT *
28 FROM mv_enrollment_flat
29 WHERE student_name = 'Student 11';
30
31 SELECT *
32 FROM mv_enrollment_flat
33 ORDER BY department_name, instructor_name, course_name, student_name;
34
35 -- Khalid Hussain (2022F-BCE-130)

```

**Output:**

## 01\_schema\_and\_seed.sql:

| Data Output                                                                                                                                          | Messages           | Notifications        |
|------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|----------------------|
| <div> <div>≡+</div> <div>📄</div> <div>▼</div> <div>📋</div> <div>▼</div> <div>🗑️</div> <div>🗄️</div> <div>⬇️</div> <div>📈</div> <div>SQL</div> </div> |                    |                      |
|                                                                                                                                                      | table_name<br>text | total_rows<br>bigint |
| 1                                                                                                                                                    | Departments        | 3                    |
| 2                                                                                                                                                    | Courses            | 5                    |
| 3                                                                                                                                                    | Instructors        | 5                    |
| 4                                                                                                                                                    | Enrollments        | 10                   |

02\_materialized\_view\_and\_queries.sql:

| Data Output Messages Notifications |                                        |                                            |                                    |
|------------------------------------|----------------------------------------|--------------------------------------------|------------------------------------|
|                                    | course_name<br>character varying (100) | department_name<br>character varying (100) | unique_students_enrolled<br>bigint |
| 1                                  | Big Data Analytics                     | Computer Science                           | 1                                  |
| 2                                  | Data Warehousing                       | Data Science                               | 4                                  |
| 3                                  | Database Systems                       | Computer Science                           | 2                                  |
| 4                                  | Digital Logic Design                   | Electrical Engineering                     | 1                                  |
| 5                                  | Machine Learning                       | Data Science                               | 2                                  |

03\_refresh\_demo.sql:

| Data Output Messages Notifications |                                         |                                        |                    |                                            |                                            |
|------------------------------------|-----------------------------------------|----------------------------------------|--------------------|--------------------------------------------|--------------------------------------------|
|                                    | student_name<br>character varying (100) | course_name<br>character varying (100) | credits<br>integer | instructor_name<br>character varying (100) | department_name<br>character varying (100) |
| 1                                  | Khalid Hussain                          | Database Systems                       | 3                  | Miss Arifa                                 | Computer Science                           |
| 2                                  | Sheikh Khalid                           | Database Systems                       | 3                  | Miss Arifa                                 | Computer Science                           |
| 3                                  | Student 11                              | Big Data Analytics                     | 3                  | Teacher 2                                  | Computer Science                           |
| 4                                  | Student 7                               | Big Data Analytics                     | 3                  | Teacher 2                                  | Computer Science                           |
| 5                                  | Sheikh Khalid Hussain                   | Data Warehousing                       | 4                  | Teacher 3                                  | Data Science                               |
| 6                                  | Student 10                              | Data Warehousing                       | 4                  | Teacher 3                                  | Data Science                               |
| 7                                  | Student 4                               | Data Warehousing                       | 4                  | Teacher 3                                  | Data Science                               |
| 8                                  | Student 9                               | Data Warehousing                       | 4                  | Teacher 3                                  | Data Science                               |
| 9                                  | Student 5                               | Machine Learning                       | 4                  | Teacher 4                                  | Data Science                               |
| 10                                 | Student 6                               | Machine Learning                       | 4                  | Teacher 4                                  | Data Science                               |
| 11                                 | Student 8                               | Digital Logic Design                   | 3                  | Teacher 5                                  | Electrical Engineering                     |

**Task 5:**

Using the provided sample data before adding the new student:

1. Data Science: 24 total credits
2. Computer Science: 9 total credits
3. Electrical Engineering: 3 total credits

**Task 6:**

Top three faculty members handling the most students:

1. Teacher 3, Data Science: 4 students
2. Miss Arifa, Computer Science: 2 students

3. Teacher 4, Data Science: 2 students

Note: The second and third positions are tied on student count. The SQL orders ties alphabetically by instructor name.

**Task 7:**

Unique students per course before adding the new student:

1. Big Data Analytics, Computer Science: 1
2. Data Warehousing, Data Science: 4
3. Database Systems, Computer Science: 2
4. Digital Logic Design, Electrical Engineering: 1
5. Machine Learning, Data Science: 2

**Task 8:**

After inserting the new enrollment into `enrollments`, the new student does not appear in `mv\_enrollment\_flat` immediately. That happens because a materialized view stores a snapshot of data taken at the time it was created or last refreshed.

**Task 9:**

After running `REFRESH MATERIALIZED VIEW mv\_enrollment\_flat;`, the new student appears in the materialized view because PostgreSQL rebuilds the stored snapshot using the latest data from the base tables.

**Task 10:**

In a real-world Big Data environment, refreshing after every new row can be expensive because rebuilding the materialized view repeatedly consumes CPU, memory, disk I/O, and can slow down transactional inserts. Refreshing once at night, such as 2 AM, is usually better because:

- reporting workloads are often lower at that time,
- batch refresh is more efficient than constant refreshes,
- users still get consistent daily reporting data,
- OLTP inserts remain faster during busy hours.

## LAB # 4

**Objective: Construct** a Star Schema by using Fact and Dimension tables

### 4.3. EXERCISES

For all tasks, use the Snipping Tool to take focused screenshots. Your screenshots should be clear and readable.

1. Create a Star Schema for the *university\_dw* created in Lab 03, that answers the following BI questions. Clearly identify Facts and Dimensions.
  - a. The University policy is to allocate the budget based on the actual teaching load of a department. Rank the departments in order of total credits.
  - b. The University wants to determine if any faculty member is “overloaded”. Identify the top three busiest faculty members and their departments.
  - c. The University wants to allocate classrooms based on the number of students enrolled in a course. For each course, list the unique number of students enrolled and the department overseeing it.
2. What is the “Grain” of the Fact table?
3. Write queries to answer the BI questions.
4. Use the DBeaver Entity Relationship Diagram (ERD) tool to visualize the Star Schema.
5. Use the DBeaver filter UI (not a manual SQL script) to show only students enrolled in Course ID 3. Count the rows displayed in the bottom status bar. Does this match your query result?

### Source Code:

01\_source\_schema\_and\_seed:

```

Query  Query History
1      -- Khalid Hussain (2022F-BCE-130)
2      -- Rebuild the normalized source schema from Lab 3.
3
4      DROP MATERIALIZED VIEW IF EXISTS mv_enrollment_flat;
5      DROP TABLE IF EXISTS enrollments;
6      DROP TABLE IF EXISTS instructors;
7      DROP TABLE IF EXISTS courses;
8      DROP TABLE IF EXISTS departments;
9
10     CREATE TABLE departments (
11         department_id INT PRIMARY KEY,
12         department_name VARCHAR(100) NOT NULL
13     );
14
15     CREATE TABLE courses (
16         course_id INT PRIMARY KEY,
17         course_name VARCHAR(100) NOT NULL,
18         credits INT NOT NULL CHECK (credits > 0)
19     );
20
21     CREATE TABLE instructors (
22         instructor_id INT PRIMARY KEY,
23         instructor_name VARCHAR(100) NOT NULL,
24         department_id INT NOT NULL REFERENCES departments(department_id)
25     );
26
  
```

```
27 CREATE TABLE enrollments (  
28     enrollment_id INT PRIMARY KEY,  
29     student_name VARCHAR(100) NOT NULL,  
30     course_id INT NOT NULL REFERENCES courses(course_id),  
31     instructor_id INT NOT NULL REFERENCES instructors(instructor_id)  
32 );  
33  
34 INSERT INTO departments (department_id, department_name) VALUES  
35     (1, 'Computer Science'),  
36     (2, 'Data Science'),  
37     (3, 'Electrical Engineering');  
38  
39 INSERT INTO courses (course_id, course_name, credits) VALUES  
40     (101, 'Database Systems', 3),  
41     (102, 'Data Warehousing', 4),  
42     (103, 'Machine Learning', 4),  
43     (104, 'Big Data Analytics', 3),  
44     (105, 'Digital Logic Design', 3);  
45  
46 INSERT INTO instructors (instructor_id, instructor_name, department_id) VALUES  
47     (1, 'Miss Arifa', 1),  
48     (2, 'Teacher 2', 1),  
49     (3, 'Teacher 3', 2),  
50     (4, 'Teacher 4', 2),  
51     (5, 'Teacher 5', 3);  
52  
53 INSERT INTO enrollments (enrollment_id, student_name, course_id, instructor_id) VALUES  
54     (1, 'Sheikh Khalid', 101, 1),  
55     (2, 'Khalid Hussain', 101, 1),  
56     (3, 'Sheikh Khalid Hussain', 102, 3),  
57     (4, 'Student 4', 102, 3),  
58     (5, 'Student 5', 103, 4),  
59     (6, 'Student 6', 103, 4),  
60     (7, 'Student 7', 104, 2),  
61     (8, 'Student 8', 105, 5),  
62     (9, 'Student 9', 102, 3),  
63     (10, 'Student 10', 102, 3);  
64  
65 SELECT 'departments' AS table_name, COUNT(*) AS total_rows FROM departments  
66 UNION ALL  
67 SELECT 'courses', COUNT(*) FROM courses  
68 UNION ALL  
69 SELECT 'instructors', COUNT(*) FROM instructors  
70 UNION ALL  
71 SELECT 'enrollments', COUNT(*) FROM enrollments;  
72  
73 -- Khalid Hussain (2022F-BCE-130)
```

02\_create\_star\_schema:

| Query | Query History                                                   |
|-------|-----------------------------------------------------------------|
| 1     | -- Khalid Hussain (2022F-BCE-130)                               |
| 2     | -- Build a star schema from the Lab 3 normalized source tables. |
| 3     |                                                                 |
| 4     | DROP TABLE IF EXISTS fact_enrollment;                           |
| 5     | DROP TABLE IF EXISTS dim_student;                               |
| 6     | DROP TABLE IF EXISTS dim_course;                                |
| 7     | DROP TABLE IF EXISTS dim_instructor;                            |
| 8     | DROP TABLE IF EXISTS dim_department;                            |
| 9     |                                                                 |
| 10    | CREATE TABLE dim_student (                                      |
| 11    | student_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,        |
| 12    | student_name VARCHAR(100) NOT NULL UNIQUE                       |
| 13    | );                                                              |
| 14    |                                                                 |
| 15    | CREATE TABLE dim_course (                                       |
| 16    | course_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,         |
| 17    | source_course_id INT NOT NULL UNIQUE,                           |
| 18    | course_name VARCHAR(100) NOT NULL,                              |
| 19    | catalog_credits INT NOT NULL CHECK (catalog_credits > 0)        |
| 20    | );                                                              |
| 21    |                                                                 |
| 22    | CREATE TABLE dim_instructor (                                   |
| 23    | instructor_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,     |
| 24    | source_instructor_id INT NOT NULL UNIQUE,                       |
| 25    | instructor_name VARCHAR(100) NOT NULL                           |
| 26    | );                                                              |

| Query | Query History                                                                  |
|-------|--------------------------------------------------------------------------------|
| 27    |                                                                                |
| 28    | CREATE TABLE dim_department (                                                  |
| 29    | department_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,                    |
| 30    | source_department_id INT NOT NULL UNIQUE,                                      |
| 31    | department_name VARCHAR(100) NOT NULL UNIQUE                                   |
| 32    | );                                                                             |
| 33    |                                                                                |
| 34    | CREATE TABLE fact_enrollment (                                                 |
| 35    | enrollment_fact_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,            |
| 36    | source_enrollment_id INT NOT NULL UNIQUE,                                      |
| 37    | student_id INT NOT NULL REFERENCES dim_student(student_id),                    |
| 38    | course_id INT NOT NULL REFERENCES dim_course(course_id),                       |
| 39    | instructor_id INT NOT NULL REFERENCES dim_instructor(instructor_id),           |
| 40    | department_id INT NOT NULL REFERENCES dim_department(department_id),           |
| 41    | credits INT NOT NULL CHECK (credits > 0),                                      |
| 42    | enrollment_count INT NOT NULL DEFAULT 1 CHECK (enrollment_count = 1)           |
| 43    | );                                                                             |
| 44    |                                                                                |
| 45    | CREATE INDEX idx_fact_enrollment_student ON fact_enrollment(student_id);       |
| 46    | CREATE INDEX idx_fact_enrollment_course ON fact_enrollment(course_id);         |
| 47    | CREATE INDEX idx_fact_enrollment_instructor ON fact_enrollment(instructor_id); |
| 48    | CREATE INDEX idx_fact_enrollment_department ON fact_enrollment(department_id); |
| 49    |                                                                                |
| 50    | INSERT INTO dim_department (source_department_id, department_name)             |
| 51    | SELECT                                                                         |
| 52    | d.department_id,                                                               |

| Query | Query History                                                           |
|-------|-------------------------------------------------------------------------|
| 53    | d.department_name                                                       |
| 54    | FROM departments d                                                      |
| 55    | ORDER BY d.department_id;                                               |
| 56    |                                                                         |
| 57    | INSERT INTO dim_course (source_course_id, course_name, catalog_credits) |
| 58    | SELECT                                                                  |
| 59    | c.course_id,                                                            |
| 60    | c.course_name,                                                          |
| 61    | c.credits                                                               |
| 62    | FROM courses c                                                          |
| 63    | ORDER BY c.course_id;                                                   |
| 64    |                                                                         |
| 65    | INSERT INTO dim_instructor (source_instructor_id, instructor_name)      |
| 66    | SELECT                                                                  |
| 67    | i.instructor_id,                                                        |
| 68    | i.instructor_name                                                       |
| 69    | FROM instructors i                                                      |
| 70    | ORDER BY i.instructor_id;                                               |
| 71    |                                                                         |
| 72    | INSERT INTO dim_student (student_name)                                  |
| 73    | SELECT x.student_name                                                   |
| 74    | FROM (                                                                  |
| 75    | SELECT                                                                  |
| 76    | e.student_name,                                                         |
| 77    | MIN(e.enrollment_id) AS first_enrollment_id                             |
| 78    | FROM enrollments e                                                      |

| Query | Query History                                   |
|-------|-------------------------------------------------|
| 79    | GROUP BY e.student_name                         |
| 80    | ) AS x                                          |
| 81    | ORDER BY x.first_enrollment_id, x.student_name; |
| 82    |                                                 |
| 83    | INSERT INTO fact_enrollment (                   |
| 84    | source_enrollment_id,                           |
| 85    | student_id,                                     |
| 86    | course_id,                                      |
| 87    | instructor_id,                                  |
| 88    | department_id,                                  |
| 89    | credits,                                        |
| 90    | enrollment_count                                |
| 91    | )                                               |
| 92    | SELECT                                          |
| 93    | e.enrollment_id,                                |
| 94    | ds.student_id,                                  |
| 95    | dc.course_id,                                   |
| 96    | di.instructor_id,                               |
| 97    | dd.department_id,                               |
| 98    | c.credits,                                      |
| 99    | 1                                               |
| 100   | FROM enrollments e                              |
| 101   | JOIN courses c                                  |
| 102   | ON c.course_id = e.course_id                    |
| 103   | JOIN instructors i                              |
| 104   | ON i.instructor_id = e.instructor_id            |

```

105 JOIN departments d
106     ON d.department_id = i.department_id
107 JOIN dim_student ds
108     ON ds.student_name = e.student_name
109 JOIN dim_course dc
110     ON dc.source_course_id = c.course_id
111 JOIN dim_instructor di
112     ON di.source_instructor_id = i.instructor_id
113 JOIN dim_department dd
114     ON dd.source_department_id = d.department_id
115 ORDER BY e.enrollment_id;
116
117 SELECT 'dim_student' AS table_name, COUNT(*) AS total_rows FROM dim_student
118 UNION ALL
119 SELECT 'dim_course', COUNT(*) FROM dim_course
120 UNION ALL
121 SELECT 'dim_instructor', COUNT(*) FROM dim_instructor
122 UNION ALL
123 SELECT 'dim_department', COUNT(*) FROM dim_department
124 UNION ALL
125 SELECT 'fact_enrollment', COUNT(*) FROM fact_enrollment;
126
127 -- Khalid Hussain (2022F-BCE-130)

```

03\_bi\_queries:

| Query | Query History                                                                       |
|-------|-------------------------------------------------------------------------------------|
| 1     | -- Khalid Hussain (2022F-BCE-130)                                                   |
| 2     | -- Task 1(a): Rank departments by total credits handled.                            |
| 3     | SELECT                                                                              |
| 4     | DENSE_RANK() OVER (ORDER BY SUM(f.credits) DESC) AS department_rank,                |
| 5     | d.department_name,                                                                  |
| 6     | SUM(f.credits) AS total_credits                                                     |
| 7     | FROM fact_enrollment f                                                              |
| 8     | JOIN dim_department d                                                               |
| 9     | ON d.department_id = f.department_id                                                |
| 10    | GROUP BY d.department_name                                                          |
| 11    | ORDER BY department_rank, d.department_name;                                        |
| 12    |                                                                                     |
| 13    | -- Task 1(b): Identify the top three busiest faculty members and their departments. |
| 14    | SELECT                                                                              |
| 15    | i.instructor_name,                                                                  |
| 16    | d.department_name,                                                                  |
| 17    | SUM(f.enrollment_count) AS students_handled                                         |
| 18    | FROM fact_enrollment f                                                              |
| 19    | JOIN dim_instructor i                                                               |
| 20    | ON i.instructor_id = f.instructor_id                                                |
| 21    | JOIN dim_department d                                                               |
| 22    | ON d.department_id = f.department_id                                                |
| 23    | GROUP BY i.instructor_name, d.department_name                                       |
| 24    | ORDER BY students_handled DESC, i.instructor_name                                   |
| 25    | LIMIT 3;                                                                            |

```
Query  Query History
26
27 -- Task 1(c): For each course, list the unique students enrolled and the overseeing department
28 SELECT
29     c.course_name,
30     d.department_name,
31     COUNT(DISTINCT s.student_id) AS unique_students_enrolled
32 FROM fact_enrollment f
33 JOIN dim_student s
34     ON s.student_id = f.student_id
35 JOIN dim_course c
36     ON c.course_id = f.course_id
37 JOIN dim_department d
38     ON d.department_id = f.department_id
39 GROUP BY c.course_name, d.department_name
40 ORDER BY c.course_name, d.department_name;
41
42 -- Support query: show the surrogate course IDs used in the warehouse.
43 SELECT
44     course_id,
45     source_course_id,
46     course_name,
47     catalog_credits
48 FROM dim_course
49 ORDER BY course_id;
50
```

```
Query  Query History
51 -- Task 5 support: filter Course ID 3 in SQL to match the DBeaver UI filter.
52 SELECT
53     f.source_enrollment_id,
54     s.student_name,
55     c.course_id,
56     c.source_course_id,
57     c.course_name,
58     i.instructor_name,
59     d.department_name
60 FROM fact_enrollment f
61 JOIN dim_student s
62     ON s.student_id = f.student_id
63 JOIN dim_course c
64     ON c.course_id = f.course_id
65 JOIN dim_instructor i
66     ON i.instructor_id = f.instructor_id
67 JOIN dim_department d
68     ON d.department_id = f.department_id
69 WHERE f.course_id = 3
70 ORDER BY f.source_enrollment_id;
71
72 -- Task 5 answer query: the DBeaver filter count should be 2.
73 SELECT COUNT(*) AS rows_for_course_id_3
74 FROM fact_enrollment
75 WHERE course_id = 3;
```

**Output:**

01\_source\_schema\_and\_seed:

| Data Output |                    |                      | Messages | Notifications |
|-------------|--------------------|----------------------|----------|---------------|
|             | table_name<br>text | total_rows<br>bigint |          |               |
| 1           | departments        | 3                    |          |               |
| 2           | courses            | 5                    |          |               |
| 3           | instructors        | 5                    |          |               |
| 4           | enrollments        | 10                   |          |               |

02\_create\_star\_schema:

| Data Output |                    |                      | Messages | Notifications |
|-------------|--------------------|----------------------|----------|---------------|
|             | table_name<br>text | total_rows<br>bigint |          |               |
| 1           | dim_student        | 10                   |          |               |
| 2           | dim_course         | 5                    |          |               |
| 3           | dim_instructor     | 5                    |          |               |
| 4           | dim_departm...     | 3                    |          |               |
| 5           | fact_enrollme...   | 10                   |          |               |

03\_bi\_queries:

| Data Output |                                |  | Messages | Notifications |
|-------------|--------------------------------|--|----------|---------------|
|             | rows_for_course_id_3<br>bigint |  |          |               |
| 1           | 2                              |  |          |               |

Task 1(a):

Rank departments in order of total credits:

| Rank | Department             | Total Credits |
|------|------------------------|---------------|
| 1    | Data Science           | 24            |
| 2    | Computer Science       | 9             |
| 3    | Electrical Engineering | 3             |

**Task 1(b):**

Top three busiest faculty members and their departments:

| Instructor | Department       | Students Handled |
|------------|------------------|------------------|
| Teacher 3  | Data Science     | 4                |
| Miss Arifa | Computer Science | 2                |
| Teacher 4  | Data Science     | 2                |

**Task 1(c):**

Unique students enrolled in each course and the overseeing department:

| Course               | Department             | Unique Students |
|----------------------|------------------------|-----------------|
| Big Data Analytics   | Computer Science       | 1               |
| Data Warehousing     | Data Science           | 4               |
| Database Systems     | Computer Science       | 2               |
| Digital Logic Design | Electrical Engineering | 1               |
| Machine Learning     | Data Science           | 2               |

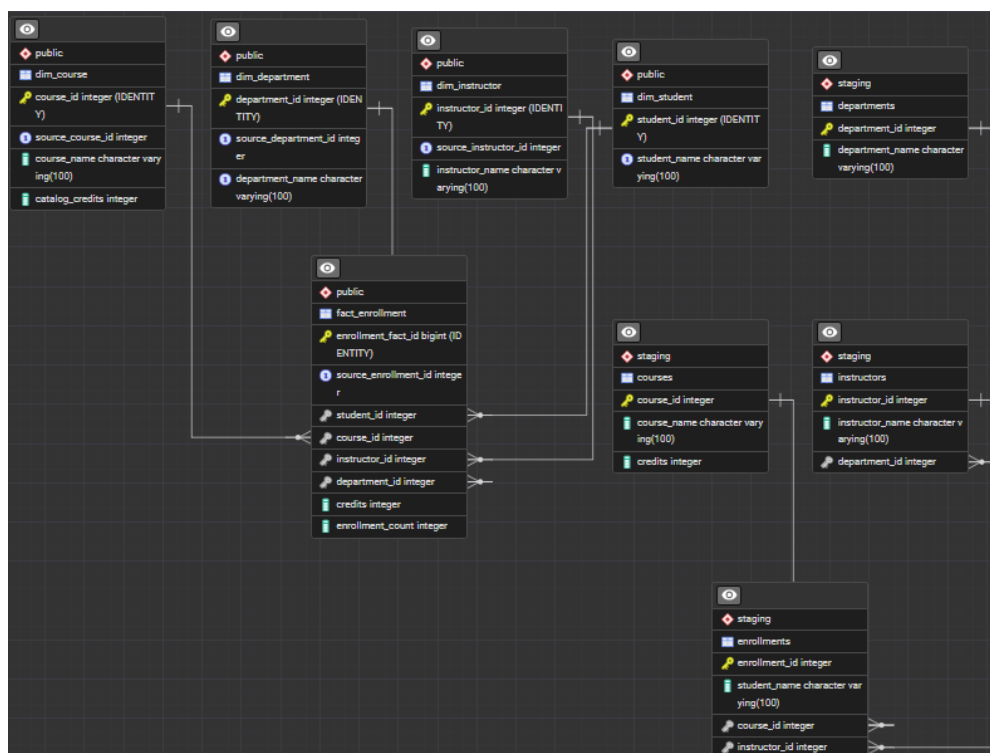
**Task 5:**

Using the warehouse surrogate keys, `course\_id = 3` maps to:

- `source\_course\_id = 103`
- `course\_name = Machine Learning`

Students enrolled in Course ID 3:

- Student 5
- Student 6

**Star Schema:**

## LAB 5

### 5.1. OBJECTIVE

Implement Slowly Changing Dimensions (SCD) logic for maintaining data history

### 5.3. EXERCISES

For all tasks, use the Snipping Tool to take focused screenshots. Your screenshots should be clear and readable.

#### 5.3.1. Task 01

Work with the Star Schema for the *university\_dw* created in Lab 04 to implement SCD.

1. Add an instructor surrogate key to the instructor dimension.
2. Link the fact table to the instructor dimension using the surrogate key.
3. Modify the structure of the instructor dimension to implement SCD type 2.
4. You noticed a typo. An instructor's name was entered incorrectly. Use SCD Type 1 to make corrections.
5. On March 02, 2026, Instructor ID 2 officially moves from the Computer Science department to the Artificial Intelligence department. Expire the old version and insert the new version in the DW.
6. The Dean is looking at a budget report from February 2026. At that time, Instructor ID 2 was still in the Computer Science department. Write a query to find which Department Name Instructor ID 2 belonged to on February 15th, 2026.
7. In your *Enrollments* table, should the *Instructor\_ID* column now point to the *Instructor\_ID* (Business Key) or the *Instructor\_SK* (Surrogate Key)? Explain your reasoning.

01\_source\_schema\_and\_seed:

```

1  -- Rebuild the normalized Lab 3 source schema in a separate staging schema
2  -- so the warehouse tables in public stay clean for reporting and screenshots.
3  -- Khalid Hussain (2022F-BCE-130)
4
5  DROP MATERIALIZED VIEW IF EXISTS public.mv_enrollment_flat;
6  DROP TABLE IF EXISTS public.enrollments;
7  DROP TABLE IF EXISTS public.instructors;
8  DROP TABLE IF EXISTS public.courses;
9  DROP TABLE IF EXISTS public.departments;
10
11 DROP SCHEMA IF EXISTS staging CASCADE;
12 CREATE SCHEMA staging;
13
14 CREATE TABLE staging.departments (
15     department_id INT PRIMARY KEY,
16     department_name VARCHAR(100) NOT NULL
17 );
18
19 CREATE TABLE staging.courses (
20     course_id INT PRIMARY KEY,
21     course_name VARCHAR(100) NOT NULL,
22     credits INT NOT NULL CHECK (credits > 0)
23 );
24
25 CREATE TABLE staging.instructors (
26     instructor_id INT PRIMARY KEY,
27     instructor_name VARCHAR(100) NOT NULL,
28     department_id INT NOT NULL REFERENCES staging.departments(department_id)
29 );
30
31 CREATE TABLE staging.enrollments (
32     enrollment_id INT PRIMARY KEY,
33     student_name VARCHAR(100) NOT NULL,
34     course_id INT NOT NULL REFERENCES staging.courses(course_id),
35     instructor_id INT NOT NULL REFERENCES staging.instructors(instructor_id)
36 );
37
38 INSERT INTO staging.departments (department_id, department_name) VALUES
39     (1, 'Computer Science'),
40     (2, 'Data Science'),
41     (3, 'Electrical Engineering');
42
43 INSERT INTO staging.courses (course_id, course_name, credits) VALUES
44     (101, 'Database Systems', 3),
45     (102, 'Data Warehousing', 4),
46     (103, 'Machine Learning', 4),
47     (104, 'Big Data Analytics', 3),
48     (105, 'Digital Logic Design', 3);
49
50 INSERT INTO staging.instructors (instructor_id, instructor_name, department_id) VALUES
51     (1, 'Miss Arifa', 1),
52     (2, 'Teacher 2', 1),
53     (3, 'Teacher 3', 2),
54     (4, 'Teacher 4', 2),
55     (5, 'Teacher 5', 3);
56
57 INSERT INTO staging.enrollments (enrollment_id, student_name, course_id, instructor_id) VALUES
58     (1, 'Sheikh Khalid', 101, 1),
59     (2, 'Khalid Hussain', 101, 1),
60     (3, 'Sheikh Khalid Hussain', 102, 3),
61     (4, 'Student 4', 102, 3),
62     (5, 'Student 5', 103, 4),
63     (6, 'Student 6', 103, 4),
64     (7, 'Student 7', 104, 2),
65     (8, 'Student 8', 105, 5),
66     (9, 'Student 9', 102, 3),
67     (10, 'Student 10', 102, 3);
68
69 SELECT 'staging.departments' AS table_name, COUNT(*) AS total_rows FROM staging.departments
70 UNION ALL
71 SELECT 'staging.courses', COUNT(*) FROM staging.courses
72 UNION ALL
73 SELECT 'staging.instructors', COUNT(*) FROM staging.instructors
74 UNION ALL
75 SELECT 'staging.enrollments', COUNT(*) FROM staging.enrollments;

```

## Output:

|   | table_name<br>text  | total_rows<br>bigint |
|---|---------------------|----------------------|
| 1 | staging.departme... | 3                    |
| 2 | staging.courses     | 5                    |
| 3 | staging.instructors | 5                    |
| 4 | staging.enrollme... | 10                   |

## 02\_create\_star\_schema:

```

1  -- Recreate the Lab 4 star schema first, then Lab 5 will upgrade the
2  -- instructor dimension into an SCD Type 2 dimension.
3  -- Khalid Hussain (2022F-BCE-130)
4
5  DROP TABLE IF EXISTS public.fact_enrollment;
6  DROP TABLE IF EXISTS public.dim_student;
7  DROP TABLE IF EXISTS public.dim_course;
8  DROP TABLE IF EXISTS public.dim_instructor;
9  DROP TABLE IF EXISTS public.dim_department;
10
11 CREATE TABLE public.dim_student (
12     student_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
13     student_name VARCHAR(100) NOT NULL UNIQUE
14 );
15
16 CREATE TABLE public.dim_course (
17     course_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
18     source_course_id INT NOT NULL UNIQUE,
19     course_name VARCHAR(100) NOT NULL,
20     catalog_credits INT NOT NULL CHECK (catalog_credits > 0)
21 );
22
23 CREATE TABLE public.dim_instructor (
24     instructor_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
25     source_instructor_id INT NOT NULL UNIQUE,
26     instructor_name VARCHAR(100) NOT NULL
27 );
28
29 CREATE TABLE public.dim_department (
30     department_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
31     source_department_id INT NOT NULL UNIQUE,
32     department_name VARCHAR(100) NOT NULL UNIQUE
33 );
34
35 CREATE TABLE public.fact_enrollment (
36     enrollment_fact_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
37     source_enrollment_id INT NOT NULL UNIQUE,
38     student_id INT NOT NULL REFERENCES public.dim_student(student_id),
39     course_id INT NOT NULL REFERENCES public.dim_course(course_id),
40     instructor_id INT NOT NULL REFERENCES public.dim_instructor(instructor_id),
41     department_id INT NOT NULL REFERENCES public.dim_department(department_id),
42     credits INT NOT NULL CHECK (credits > 0),
43     enrollment_count INT NOT NULL DEFAULT 1 CHECK (enrollment_count = 1)
44 );
45
46 CREATE INDEX idx_fact_enrollment_student ON public.fact_enrollment(student_id);
47 CREATE INDEX idx_fact_enrollment_course ON public.fact_enrollment(course_id);
48 CREATE INDEX idx_fact_enrollment_instructor ON public.fact_enrollment(instructor_id);
49 CREATE INDEX idx_fact_enrollment_department ON public.fact_enrollment(department_id);
50
51 INSERT INTO public.dim_department (source_department_id, department_name)
52 SELECT
53     d.department_id,
54     d.department_name
55 FROM staging.departments d
56 ORDER BY d.department_id;
57
58 INSERT INTO public.dim_course (source_course_id, course_name, catalog_credits)
59 SELECT
60     c.course_id,
61     c.course_name,
62     c.credits
63 FROM staging.courses c
64 ORDER BY c.course_id;
65
66 INSERT INTO public.dim_instructor (source_instructor_id, instructor_name)
67 SELECT
68     i.instructor_id,
69     i.instructor_name
70 FROM staging.instructors i
71 ORDER BY i.instructor_id;
72
73 INSERT INTO public.dim_student (student_name)
74 SELECT x.student_name
75 FROM (
76     SELECT
77         e.student_name,
78         MIN(e.enrollment_id) AS first_enrollment_id
79     FROM staging.enrollments e
80     GROUP BY e.student_name
81 ) AS x
82 ORDER BY x.first_enrollment_id, x.student_name;

```

```

83
84 INSERT INTO public.fact_enrollment (
85     source_enrollment_id,
86     student_id,
87     course_id,
88     instructor_id,
89     department_id,
90     credits,
91     enrollment_count
92 )
93 SELECT
94     e.enrollment_id,
95     ds.student_id,
96     dc.course_id,
97     di.instructor_id,
98     dd.department_id,
99     c.credits,
100     1
101 FROM staging.enrollments e
102 JOIN staging.courses c
103     ON c.course_id = e.course_id
104 JOIN staging.instructors i
105     ON i.instructor_id = e.instructor_id
106 JOIN staging.departments d
107     ON d.department_id = i.department_id
108 JOIN public.dim_student ds
109     ON ds.student_name = e.student_name
110 JOIN public.dim_course dc
111     ON dc.source_course_id = c.course_id
112 JOIN public.dim_instructor di
113     ON di.source_instructor_id = i.instructor_id
114 JOIN public.dim_department dd
115     ON dd.source_department_id = d.department_id
116 ORDER BY e.enrollment_id;
117
118 SELECT 'dim_student' AS table_name, COUNT(*) AS total_rows FROM dim_student
119 UNION ALL
120 SELECT 'dim_course', COUNT(*) FROM dim_course
121 UNION ALL
122 SELECT 'dim_instructor', COUNT(*) FROM dim_instructor
123 UNION ALL
124 SELECT 'dim_department', COUNT(*) FROM dim_department
125 UNION ALL
126 SELECT 'fact_enrollment', COUNT(*) FROM fact_enrollment;

```

### Output:

|   | table_name<br>text | total_rows<br>bigint |
|---|--------------------|----------------------|
| 1 | dim_student        | 10                   |
| 2 | dim_course         | 5                    |
| 3 | dim_instructor     | 5                    |
| 4 | dim_department     | 3                    |
| 5 | fact_enrollment    | 10                   |

### 03\_apply\_scd\_logic:

```

1  -- Upgrade the Lab 4 warehouse to satisfy the Lab 5 SCD tasks.
2  -- Khalid Hussain (2022F-BCE-130)
3
4  BEGIN;
5
6  -- Task 1 and Task 2:
7  -- Make the instructor surrogate key explicit and point the fact table to it.
8  ALTER TABLE public.fact_enrollment
9      DROP CONSTRAINT IF EXISTS fact_enrollment_instructor_id_fkey;
10
11 ALTER TABLE public.dim_instructor
12     RENAME COLUMN instructor_id TO instructor_sk;
13
14 ALTER TABLE public.dim_instructor
15     RENAME COLUMN source_instructor_id TO instructor_id;
16
17 ALTER TABLE public.fact_enrollment
18     RENAME COLUMN instructor_id TO instructor_sk;
19
20 DROP INDEX IF EXISTS public.idx_fact_enrollment_instructor;
21 CREATE INDEX idx_fact_enrollment_instructor_sk ON public.fact_enrollment(instructor_sk);
22
23 ALTER TABLE public.dim_instructor
24     DROP CONSTRAINT IF EXISTS dim_instructor_source_instructor_id_key;
25
26 ALTER TABLE public.fact_enrollment
27     ADD CONSTRAINT fact_enrollment_instructor_sk_fkey
28     FOREIGN KEY (instructor_sk) REFERENCES public.dim_instructor(instructor_sk);
29
30 -- Task 3:
31 -- Add the department reference and SCD Type 2 tracking columns.
32 ALTER TABLE public.dim_instructor
33     ADD COLUMN department_id INT;
34
35 UPDATE public.dim_instructor di
36 SET department_id = dd.department_id
37 FROM staging.instructors si
38 JOIN public.dim_department dd
39     ON dd.source_department_id = si.department_id
40 WHERE di.instructor_id = si.instructor_id;
41

```

```

41
42 ALTER TABLE public.dim_instructor
43     ALTER COLUMN department_id SET NOT NULL,
44     ADD COLUMN start_date DATE NOT NULL DEFAULT DATE '2020-01-01',
45     ADD COLUMN end_date DATE NOT NULL DEFAULT DATE '9999-12-31',
46     ADD COLUMN is_current BOOLEAN NOT NULL DEFAULT TRUE;
47
48 ALTER TABLE public.dim_instructor
49     ADD CONSTRAINT dim_instructor_department_id_fkey
50     FOREIGN KEY (department_id) REFERENCES public.dim_department(department_id),
51     ADD CONSTRAINT dim_instructor_valid_date_range_chk
52     CHECK (start_date <= end_date);
53
54 CREATE UNIQUE INDEX uq_dim_instructor_business_key_start_date
55     ON public.dim_instructor(instructor_id, start_date);
56
57 CREATE UNIQUE INDEX uq_dim_instructor_current_version
58     ON public.dim_instructor(instructor_id)
59     WHERE is_current;
60
61 CREATE INDEX idx_dim_instructor_history_lookup
62     ON public.dim_instructor(instructor_id, start_date, end_date);
63
64 -- Task 4:
65 -- The Lab 4 seed had no typo, so the first update simulates one.
66 UPDATE public.dim_instructor
67 SET instructor_name = 'Teacher 4'
68 WHERE instructor_id = 4
69     AND is_current = TRUE;
70
71 -- SCD Type 1 correction: overwrite the current row, no new history row.
72 UPDATE public.dim_instructor
73 SET instructor_name = 'Teacher 4'
74 WHERE instructor_id = 4
75     AND is_current = TRUE;
76
77 -- Task 5:
78 -- Instructor ID 2 moves from Computer Science to Artificial Intelligence
79 -- on March 02, 2020. Expire the old row and insert the new current row.
80 INSERT INTO public.dim_department (source_department_id, department_name)
81 VALUES (4, 'Artificial Intelligence');
82

```

```

82
83 UPDATE public.dim_instructor
84 SET end_date = DATE '2020-03-01',
85     is_current = FALSE
86 WHERE instructor_id = 2
87     AND is_current = TRUE;
88
89 INSERT INTO public.dim_instructor (
90     instructor_id,
91     instructor_name,
92     department_id,
93     start_date,
94     end_date,
95     is_current
96 )
97 SELECT
98     old_version.instructor_id,
99     old_version.instructor_name,
100     ai_department.department_id,
101     DATE '2020-03-02',
102     DATE '9999-12-31',
103     TRUE
104 FROM public.dim_instructor AS old_version
105 JOIN public.dim_department AS ai_department
106     ON ai_department.source_department_id = 4
107 WHERE old_version.instructor_id = 2
108     AND old_version.end_date = DATE '2020-03-01'
109 ORDER BY old_version.instructor_sk DESC
110 LIMIT 1;
111
112 COMMIT;
113
114 -- Final view of the instructor history after the Lab 5 changes.
115 SELECT
116     di.instructor_sk,
117     di.instructor_id,
118     di.instructor_name,
119     dd.department_name,
120     di.start_date,
121     di.end_date,
122     di.is_current

```

```

123 FROM public.dim_instructor di
124 JOIN public.dim_department dd
125     ON dd.department_id = di.department_id
126 ORDER BY di.instructor_id, di.start_date, di.instructor_sk;
127

```

**Output:**

|   | instructor_sk<br>integer | instructor_id<br>integer | instructor_name<br>character varying (100) | department_name<br>character varying (100) | start_date<br>date | end_date<br>date | is_current<br>boolean |
|---|--------------------------|--------------------------|--------------------------------------------|--------------------------------------------|--------------------|------------------|-----------------------|
| 1 | 1                        | 1                        | Miss Arifa                                 | Computer Science                           | 2026-01-01         | 9999-12-31       | true                  |
| 2 | 2                        | 2                        | Teacher 2                                  | Computer Science                           | 2026-01-01         | 2026-03-01       | false                 |
| 3 | 6                        | 2                        | Teacher 2                                  | Artificial Intelligence                    | 2026-03-02         | 9999-12-31       | true                  |
| 4 | 3                        | 3                        | Teacher 3                                  | Data Science                               | 2026-01-01         | 9999-12-31       | true                  |
| 5 | 4                        | 4                        | Teacher 4                                  | Data Science                               | 2026-01-01         | 9999-12-31       | true                  |
| 6 | 5                        | 5                        | Teacher 5                                  | Electrical Engineering                     | 2026-01-01         | 9999-12-31       | true                  |

**04\_task\_queries:**

```

1  -- Task 6:
2  -- Find the department for Instructor ID 2 on February 15, 2026.
3  -- Khalid Hussain (2022F-BCE-130)
4  SELECT
5      di.instructor_id,
6      di.instructor_name,
7      dd.department_name,
8      DATE '2026-02-15' AS as_of_date
9  FROM public.dim_instructor di
10 JOIN public.dim_department dd
11     ON dd.department_id = di.department_id
12 WHERE di.instructor_id = 2
13        AND DATE '2026-02-15' BETWEEN di.start_date AND di.end_date;
14
15 -- Support query:
16 -- Show the full instructor history in business-key order.
17 SELECT
18     di.instructor_sk,
19     di.instructor_id,
20     di.instructor_name,
21     dd.department_name,
22     di.start_date,
23     di.end_date,
24     di.is_current
25 FROM public.dim_instructor di
26 JOIN public.dim_department dd
27     ON dd.department_id = di.department_id
28 ORDER BY di.instructor_id, di.start_date, di.instructor_sk;
29
30 -- Support query:
31 -- Show that fact rows now point to the surrogate key and therefore keep the
32 -- historical Computer Science version for the already-loaded Instructor ID 2 facts.
33 SELECT
34     f.source_enrollment_id,
35     f.instructor_sk,
36     di.instructor_id AS business_instructor_id,
37     di.instructor_name,
38     dd.department_name,
39     di.start_date,
40     di.end_date,
41     di.is_current
42 FROM public.fact_enrollment f
43 JOIN public.dim_instructor di
44     ON di.instructor_sk = f.instructor_sk
45 JOIN public.dim_department dd
46     ON dd.department_id = di.department_id
47 ORDER BY f.source_enrollment_id;

```

**Output:**

|    | source_enrollment_id<br>integer | instructor_sk<br>integer | business_instructor_id<br>integer | instructor_name<br>character varying (100) | department_name<br>character varying (100) | start_date<br>date | end_date<br>date | is_current<br>boolean |
|----|---------------------------------|--------------------------|-----------------------------------|--------------------------------------------|--------------------------------------------|--------------------|------------------|-----------------------|
| 1  | 1                               | 1                        | 1                                 | Miss Arifa                                 | Computer Science                           | 2026-01-01         | 9999-12-31       | true                  |
| 2  | 2                               | 1                        | 1                                 | Miss Arifa                                 | Computer Science                           | 2026-01-01         | 9999-12-31       | true                  |
| 3  | 3                               | 3                        | 3                                 | Teacher 3                                  | Data Science                               | 2026-01-01         | 9999-12-31       | true                  |
| 4  | 4                               | 3                        | 3                                 | Teacher 3                                  | Data Science                               | 2026-01-01         | 9999-12-31       | true                  |
| 5  | 5                               | 4                        | 4                                 | Teacher 4                                  | Data Science                               | 2026-01-01         | 9999-12-31       | true                  |
| 6  | 6                               | 4                        | 4                                 | Teacher 4                                  | Data Science                               | 2026-01-01         | 9999-12-31       | true                  |
| 7  | 7                               | 2                        | 2                                 | Teacher 2                                  | Computer Science                           | 2026-01-01         | 2026-03-01       | false                 |
| 8  | 8                               | 5                        | 5                                 | Teacher 5                                  | Electrical Engineering                     | 2026-01-01         | 9999-12-31       | true                  |
| 9  | 9                               | 3                        | 3                                 | Teacher 3                                  | Data Science                               | 2026-01-01         | 9999-12-31       | true                  |
| 10 | 10                              | 3                        | 3                                 | Teacher 3                                  | Data Science                               | 2026-01-01         | 9999-12-31       | true                  |

**SCD Concepts Used**

- SCD Type 1: overwrite the old descriptive value when history is not important, such as fixing a typo.
- SCD Type 2: expire the old row and insert a new row when the historical version must remain available for reporting.
- Surrogate Key: an artificial warehouse key that allows multiple rows for the same business entity across time.

**Task 1**

The instructor dimension now uses:

- instructor\_sk as the surrogate primary key
- instructor\_id as the business key

This makes the warehouse key explicit and allows multiple versions for the same instructor.

**Task 2**

fact\_enrollment now references instructor\_sk, not the business key. Existing fact rows still point to the original Computer Science version of Instructor ID 2, which preserves history.

**Task 3**

The instructor dimension was extended with:

- department\_id
- start\_date
- end\_date
- is\_current

These columns turn dim\_instructor into an SCD Type 2 dimension.

**Task 4**

The Type 1 correction overwrites the current instructor name in place:

- simulated typo: Techer 4
- corrected value: Teacher 4

No extra history row is created for a simple spelling fix.

## Task 5

Instructor ID 2 changes department on 2026-03-02:

- old Computer Science row is expired with end\_date = 2026-03-01
- new Artificial Intelligence row is inserted with start\_date = 2026-03-02

That is the required SCD Type 2 behavior because the old departmental assignment must remain available for historical reports.

## Task 6

On 2026-02-15, Instructor ID 2 belonged to:

- Computer Science

The query filters by the business key and checks which instructor version was effective on that date.

## Task 7

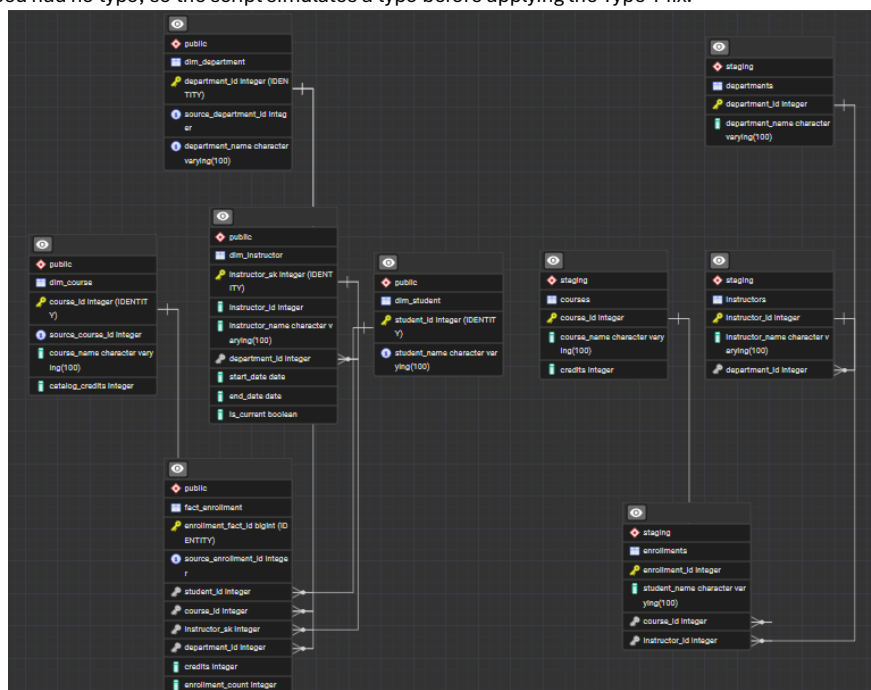
The fact table should point to Instructor\_SK, not Instructor\_ID.

Reason:

- Instructor\_ID is the business key and can have multiple historical rows in the dimension.
- Instructor\_SK identifies one exact version of that instructor.
- If the fact table used only Instructor\_ID, a join could accidentally return the current department instead of the department that was valid when the fact occurred.
- Using Instructor\_SK keeps February 2026 facts tied to the old Computer Science row even after the March 2026 move.

## Assumptions

- Initial instructor history begins on 2026-01-01.
- Artificial Intelligence is added to the warehouse department dimension with source key 4.
- The Lab 4 seed had no typo, so the script simulates a typo before applying the Type 1 fix.



## LAB 6

### 6.1. OBJECTIVE

**Setup** the ETL pipeline by extracting and transforming raw data

### 6.3. EXERCISES

For all tasks, use the Snipping Tool to take focused screenshots. Your screenshots should be clear and readable.

#### 6.3.1. Task 01

Work with the Star Schema for the *university\_dw* created in Lab 04.

The University Finance department has sent an Excel file (save as *course\_fees.csv*) containing:

| Course Name          | Base Fee |
|----------------------|----------|
| Machine Learning     | 15,000   |
| Cloud Computing      | 25,000   |
| Computer Networks    | 10,000   |
| Cyber Security       | 17,000   |
| Digital Logic Design | 18,000   |

1. Create a staging table *stg\_fees*.
2. Import the CSV data.
3. Write an UPDATE query to populate a new Fee column in your *Courses* table by joining it with *stg\_fees*.

#### 6.3.2. Task 02

You have been given raw enrollment data (save as *enrollment\_batch.csv*) which contains the student's name, course name, instructor's name, and enrollment date (At least 10 records). There is whitespace and casing issues on the names and date is in the format MM/DD/YYYY.

1. Create a staging table *stg\_enrollments*.
2. Load the CSV data into the staging table.
3. Transform the raw data and load it into the warehouse.
4. Add a row to *stg\_enrollments* with a fake course name, Quantum Physics.
5. Write a LEFT JOIN query between *stg\_enrollments* and *Courses*.
6. Identify all records in the staging table that do not have a match in the Warehouse.
7. In a real-world scenario, would you discard these records or create new entries for them?

#### 6.3.3. Task 03

1. Write the SQL command to empty the *stg\_enrollments* table without deleting its structure.
2. Explain why we TRUNCATE staging tables at the end of an ETL cycle instead of using DROP TABLE.

## 01\_source\_schema\_and\_seed:

```

1  -- Rebuild the normalized source schema used to feed the Lab 6 ETL pipeline.
2
3  DROP MATERIALIZED VIEW IF EXISTS public.mv_enrollment_flat;
4  DROP TABLE IF EXISTS public.enrollments;
5  DROP TABLE IF EXISTS public.instructors;
6  DROP TABLE IF EXISTS public.courses;
7  DROP TABLE IF EXISTS public.departments;
8
9  DROP SCHEMA IF EXISTS staging CASCADE;
10 CREATE SCHEMA staging;
11
12 CREATE OR REPLACE FUNCTION staging.normalize_name(input_text TEXT)
13 RETURNS TEXT
14 LANGUAGE sql
15 IMMUTABLE
16 AS $$
17     SELECT NULLIF(
18         INITCAP(
19             BTRIM(
20                 REGEXP_REPLACE(COALESCE(input_text, ''), '\s+', ' ', 'g')
21             )
22         ),
23         ''
24     );
25 $$;
26
27 CREATE TABLE staging.departments (
28     department_id INT PRIMARY KEY,
29     department_name VARCHAR(100) NOT NULL
30 );
31
32 CREATE TABLE staging.courses (
33     course_id INT PRIMARY KEY,
34     course_name VARCHAR(100) NOT NULL,
35     credits INT NOT NULL CHECK (credits > 0)
36 );
37
38 CREATE TABLE staging.instructors (
39     instructor_id INT PRIMARY KEY,
40     instructor_name VARCHAR(100) NOT NULL,
41     department_id INT NOT NULL REFERENCES staging.departments(department_id)
42 );
43
44 CREATE TABLE staging.enrollments (
45     enrollment_id INT PRIMARY KEY,
46     student_name VARCHAR(100) NOT NULL,
47     course_id INT NOT NULL REFERENCES staging.courses(course_id),
48     instructor_id INT NOT NULL REFERENCES staging.instructors(instructor_id)
49 );
50
51 INSERT INTO staging.departments (department_id, department_name) VALUES
52     (1, 'Computer Science'),
53     (2, 'Data Science'),
54     (3, 'Electrical Engineering');
55
56 INSERT INTO staging.courses (course_id, course_name, credits) VALUES
57     (101, 'Database Systems', 3),
58     (102, 'Data Warehousing', 4),
59     (103, 'Machine Learning', 4),
60     (104, 'Big Data Analytics', 3),
61     (105, 'Digital Logic Design', 3),
62     (106, 'Cloud Computing', 4),
63     (107, 'Computer Networks', 3),
64     (108, 'Cyber Security', 4);
65
66 INSERT INTO staging.instructors (instructor_id, instructor_name, department_id) VALUES
67     (1, 'Miss Arifa', 1),
68     (2, 'Teacher 2', 1),
69     (3, 'Teacher 3', 2),
70     (4, 'Teacher 4', 2),
71     (5, 'Teacher 5', 3);
72

```

```

73 INSERT INTO staging.enrollments (enrollment_id, student_name, course_id, instructor_id) VALUES
74     (1, 'Sheikh Khalid', 101, 1),
75     (2, 'Khalid Hussain', 101, 1),
76     (3, 'Sheikh Khalid Hussain', 102, 3),
77     (4, 'Student 4', 102, 3),
78     (5, 'Student 5', 103, 4),
79     (6, 'Student 6', 103, 4),
80     (7, 'Student 7', 104, 2),
81     (8, 'Student 8', 105, 5),
82     (9, 'Student 9', 102, 3),
83     (10, 'Student 10', 102, 3);
84
85 SELECT 'staging.departments' AS table_name, COUNT(*) AS total_rows FROM staging.departments
86 UNION ALL
87 SELECT 'staging.courses', COUNT(*) FROM staging.courses
88 UNION ALL
89 SELECT 'staging.instructors', COUNT(*) FROM staging.instructors
90 UNION ALL
91 SELECT 'staging.enrollments', COUNT(*) FROM staging.enrollments;

```

Output:

|   | table_name<br>text  | total_rows<br>bigint |
|---|---------------------|----------------------|
| 1 | staging.departme... | 3                    |
| 2 | staging.courses     | 8                    |
| 3 | staging.instructors | 5                    |
| 4 | staging.enrollme... | 10                   |

## 02\_create\_star\_schema:

```

1  -- Build the Lab 4 style star schema that Lab 6 will load through staging tables.
2
3  DROP TABLE IF EXISTS public.fact_enrollment;
4  DROP TABLE IF EXISTS public.dim_student;
5  DROP TABLE IF EXISTS public.dim_course;
6  DROP TABLE IF EXISTS public.dim_instructor;
7  DROP TABLE IF EXISTS public.dim_department;
8
9  CREATE TABLE public.dim_student (
10     student_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
11     student_name VARCHAR(100) NOT NULL UNIQUE
12 );
13
14  CREATE TABLE public.dim_course (
15     course_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
16     source_course_id INT NOT NULL UNIQUE,
17     course_name VARCHAR(100) NOT NULL,
18     catalog_credits INT NOT NULL CHECK (catalog_credits > 0)
19 );
20
21  CREATE TABLE public.dim_instructor (
22     instructor_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
23     source_instructor_id INT NOT NULL UNIQUE,
24     instructor_name VARCHAR(100) NOT NULL
25 );
26
27  CREATE TABLE public.dim_department (
28     department_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
29     source_department_id INT NOT NULL UNIQUE,
30     department_name VARCHAR(100) NOT NULL UNIQUE
31 );
32
33  CREATE TABLE public.fact_enrollment (
34     enrollment_fact_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
35     source_enrollment_id INT NOT NULL UNIQUE,
36     student_id INT NOT NULL REFERENCES public.dim_student(student_id),
37     course_id INT NOT NULL REFERENCES public.dim_course(course_id),
38     instructor_id INT NOT NULL REFERENCES public.dim_instructor(instructor_id),
39     department_id INT NOT NULL REFERENCES public.dim_department(department_id),
40     credits INT NOT NULL CHECK (credits > 0),
41     enrollment_count INT NOT NULL DEFAULT 1 CHECK (enrollment_count = 1)
42 );
43
44  CREATE INDEX idx_fact_enrollment_student ON public.fact_enrollment(student_id);
45  CREATE INDEX idx_fact_enrollment_course ON public.fact_enrollment(course_id);
46  CREATE INDEX idx_fact_enrollment_instructor ON public.fact_enrollment(instructor_id);

```

```

47 CREATE INDEX idx_fact_enrollment_department ON public.fact_enrollment(department_id);
48
49 INSERT INTO public.dim_department (source_department_id, department_name)
50 SELECT
51     d.department_id,
52     d.department_name
53 FROM staging.departments d
54 ORDER BY d.department_id;
55
56 INSERT INTO public.dim_course (source_course_id, course_name, catalog_credits)
57 SELECT
58     c.course_id,
59     c.course_name,
60     c.credits
61 FROM staging.courses c
62 ORDER BY c.course_id;
63
64 INSERT INTO public.dim_instructor (source_instructor_id, instructor_name)
65 SELECT
66     i.instructor_id,
67     i.instructor_name
68 FROM staging.instructors i
69 ORDER BY i.instructor_id;
70
71 INSERT INTO public.dim_student (student_name)
72 SELECT x.student_name
73 FROM (
74     SELECT
75         e.student_name,
76         MIN(e.enrollment_id) AS first_enrollment_id
77     FROM staging.enrollments e
78     GROUP BY e.student_name
79 ) AS x
80 ORDER BY x.first_enrollment_id, x.student_name;
81
82 INSERT INTO public.fact_enrollment (
83     source_enrollment_id,
84     student_id,
85     course_id,
86     instructor_id,
87     department_id,
88     credits,
89     enrollment_count
90 )
91 SELECT
92     e.enrollment_id,

```

```

93     ds.student_id,
94     dc.course_id,
95     di.instructor_id,
96     dd.department_id,
97     c.credits,
98     1
99 FROM staging.enrollments e
100 JOIN staging.courses c
101     ON c.course_id = e.course_id
102 JOIN staging.instructors i
103     ON i.instructor_id = e.instructor_id
104 JOIN staging.departments d
105     ON d.department_id = e.department_id
106 JOIN public.dim_student ds
107     ON ds.student_name = e.student_name
108 JOIN public.dim_course dc
109     ON dc.source_course_id = c.course_id
110 JOIN public.dim_instructor di
111     ON di.source_instructor_id = i.instructor_id
112 JOIN public.dim_department dd
113     ON dd.source_department_id = d.department_id
114 ORDER BY e.enrollment_id;
115
116 SELECT 'dim_student' AS table_name, COUNT(*) AS total_rows FROM public.dim_student
117 UNION ALL
118 SELECT 'dim_course', COUNT(*) FROM public.dim_course
119 UNION ALL
120 SELECT 'dim_instructor', COUNT(*) FROM public.dim_instructor
121 UNION ALL
122 SELECT 'dim_department', COUNT(*) FROM public.dim_department
123 UNION ALL
124 SELECT 'fact_enrollment', COUNT(*) FROM public.fact_enrollment;

```

Output:

|   | table_name      | total_rows |
|---|-----------------|------------|
|   | text            | bigint     |
| 1 | dim_student     | 10         |
| 2 | dim_course      | 8          |
| 3 | dim_instructor  | 5          |
| 4 | dim_department  | 3          |
| 5 | fact_enrollment | 10         |

## 03\_task01\_fees:

```

1  -- Task 01
2  -- 1. Create a staging table stg_fees.
3  -- 2. Import the CSV data.
4  -- 3. Update the course dimension with fee values.
5
6  DROP TABLE IF EXISTS staging.stg_fees;
7
8  CREATE TABLE staging.stg_fees (
9      course_name VARCHAR(100) NOT NULL,
10     base_fee_text VARCHAR(20) NOT NULL
11 );
12
13 INSERT INTO staging.stg_fees (course_name, base_fee_text) VALUES
14     ('Machine Learning', '15,000'),
15     ('Cloud Computing', '25,000'),
16     ('Computer Networks', '10,000'),
17     ('Cyber Security', '17,000'),
18     ('Digital Logic Design', '18,000');
19
20 ALTER TABLE public.dim_course
21     ADD COLUMN IF NOT EXISTS fee NUMERIC(12,2);
22
23 UPDATE public.dim_course dc
24 SET fee = REPLACE(sf.base_fee_text, ',', '')::NUMERIC(12,2)
25 FROM staging.stg_fees sf
26 WHERE staging.normalize_name(dc.course_name) = staging.normalize_name(sf.course_name);
27
28 SELECT
29     dc.course_id,
30     dc.course_name,
31     dc.catalog_credits,
32     dc.fee
33 FROM public.dim_course dc
34 WHERE dc.fee IS NOT NULL
35 ORDER BY dc.course_id;

```

## Output:

|   | course_id<br>[PK] integer | course_name<br>character varying (100) | catalog_credits<br>integer | fee<br>numeric (12,2) |
|---|---------------------------|----------------------------------------|----------------------------|-----------------------|
| 1 | 3                         | Machine Learning                       | 4                          | 15000.00              |
| 2 | 5                         | Digital Logic Design                   | 3                          | 18000.00              |
| 3 | 6                         | Cloud Computing                        | 4                          | 25000.00              |
| 4 | 7                         | Computer Networks                      | 3                          | 10000.00              |
| 5 | 8                         | Cyber Security                         | 4                          | 17000.00              |

## 04\_task02\_stage\_enrollments:

```

1  -- Task 02
2  -- 1. Create a staging table stg_enrollments.
3  -- 2. Load the CSV data into the staging table.
4
5  DROP TABLE IF EXISTS staging.stg_enrollments;
6
7  CREATE TABLE staging.stg_enrollments (
8      stg_enrollment_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
9      student_name_raw VARCHAR(100) NOT NULL,
10     course_name_raw VARCHAR(100) NOT NULL,
11     instructor_name_raw VARCHAR(100) NOT NULL,
12     enrollment_date_raw VARCHAR(20) NOT NULL
13 );
14
15 INSERT INTO staging.stg_enrollments (
16     student_name_raw,
17     course_name_raw,
18     instructor_name_raw,
19     enrollment_date_raw
20 ) VALUES

```

```

21 ('Ayesha Khan', 'machine learning', 'teacher 4', '02/10/2026'),
22 ('Khalid Hussain', 'CLOUD COMPUTING', 'TEACHER 3', '02/11/2026'),
23 ('SARA noor', 'computer networks', 'teacher 2', '02/12/2026'),
24 ('Sheikh Khalid', 'cyber security', 'teacher 5', '02/13/2026'),
25 ('zainab iqbal', 'digital logic design', 'Teacher 5', '02/14/2026'),
26 ('Umer Farooq', 'data warehousing', 'teacher 3', '02/15/2026'),
27 ('fatima zahra', 'database systems', 'MISS ARIFA', '02/16/2026'),
28 ('muhammad saad', 'big data analytics', 'Teacher 2', '02/17/2026'),
29 ('Noor ul ain', 'Machine Learning', 'TEACHER 4', '02/18/2026'),
30 ('Ali Raza', 'cloud computing', 'teacher 3', '02/19/2026'),
31 ('Hira Faisal', 'cyber security', 'teacher 5', '02/20/2026');
32
33 SELECT
34     stg_enrollment_id,
35     student_name_raw,
36     course_name_raw,
37     instructor_name_raw,
38     enrollment_date_raw
39 FROM staging.stg_enrollments
40 ORDER BY stg_enrollment_id;

```

Output:

|    | stg_enrollment_id<br>[PK] integer | student_name_raw<br>character varying (100) | course_name_raw<br>character varying (100) | instructor_name_raw<br>character varying (100) | enrollment_date_raw<br>character varying (20) |
|----|-----------------------------------|---------------------------------------------|--------------------------------------------|------------------------------------------------|-----------------------------------------------|
| 1  | 1                                 | Ayesha Khan                                 | machine learning                           | teacher 4                                      | 02/10/2026                                    |
| 2  | 2                                 | Khalid Hussain                              | CLOUD COMPUTING                            | TEACHER 3                                      | 02/11/2026                                    |
| 3  | 3                                 | SARA noor                                   | computer networks                          | teacher 2                                      | 02/12/2026                                    |
| 4  | 4                                 | Sheikh Khalid                               | cyber security                             | teacher 5                                      | 02/13/2026                                    |
| 5  | 5                                 | zainab iqbal                                | digital logic design                       | Teacher 5                                      | 02/14/2026                                    |
| 6  | 6                                 | Umer Farooq                                 | data warehousing                           | teacher 3                                      | 02/15/2026                                    |
| 7  | 7                                 | fatima zahra                                | database systems                           | MISS ARIFA                                     | 02/16/2026                                    |
| 8  | 8                                 | muhammad saad                               | big data analytics                         | Teacher 2                                      | 02/17/2026                                    |
| 9  | 9                                 | Noor ul ain                                 | Machine Learning                           | TEACHER 4                                      | 02/18/2026                                    |
| 10 | 10                                | Ali Raza                                    | cloud computing                            | teacher 3                                      | 02/19/2026                                    |
| 11 | 11                                | Hira Faisal                                 | cyber security                             | teacher 5                                      | 02/20/2026                                    |

#### 05\_task02\_transform\_and\_load:

```

1  -- Task 02
2  -- 3. Transform the raw data and load it into the warehouse.
3
4  ALTER TABLE public.fact_enrollment
5      ADD COLUMN IF NOT EXISTS enrollment_date DATE;
6
7  WITH cleaned_staging AS (
8      SELECT
9          se.stg_enrollment_id,
10         6000 + se.stg_enrollment_id AS source_enrollment_id,
11         staging.normalize_name(se.student_name_raw) AS student_name,
12         staging.normalize_name(se.course_name_raw) AS course_name,
13         staging.normalize_name(se.instructor_name_raw) AS instructor_name,
14         TO_DATE(BTRIM(se.enrollment_date_raw), 'MM/DD/YYYY') AS enrollment_date
15     FROM staging.stg_enrollments se
16 )
17 INSERT INTO public.dim_student (student_name)
18 SELECT DISTINCT cs.student_name
19 FROM cleaned_staging cs
20 ON CONFLICT (student_name) DO NOTHING;
21
22 WITH cleaned_staging AS (
23     SELECT
24         se.stg_enrollment_id,
25         6000 + se.stg_enrollment_id AS source_enrollment_id,
26         staging.normalize_name(se.student_name_raw) AS student_name,
27         staging.normalize_name(se.course_name_raw) AS course_name,
28         staging.normalize_name(se.instructor_name_raw) AS instructor_name,
29         TO_DATE(BTRIM(se.enrollment_date_raw), 'MM/DD/YYYY') AS enrollment_date
30     FROM staging.stg_enrollments se
31 ),

```

```

32 matched_rows AS (
33     SELECT
34         cs.source_enrollment_id,
35         ds.student_id,
36         dc.course_id,
37         di.instructor_id,
38         dd.department_id,
39         dc.catalog_credits AS credits,
40         cs.enrollment_date
41     FROM cleaned_staging cs
42     JOIN public.dim_student ds
43         ON ds.student_name = cs.student_name
44     JOIN public.dim_course dc
45         ON staging.normalize_name(dc.course_name) = cs.course_name
46     JOIN public.dim_instructor di
47         ON staging.normalize_name(di.instructor_name) = cs.instructor_name
48     JOIN staging.instructors si
49         ON si.instructor_id = di.source_instructor_id
50     JOIN public.dim_department dd
51         ON dd.source_department_id = si.department_id
52 )
53 INSERT INTO public.fact_enrollment (
54     source_enrollment_id,
55     student_id,
56     course_id,
57     instructor_id,
58     department_id,
59     credits,
60     enrollment_count,
61     enrollment_date
62 )

```

```

63 SELECT
64     mr.source_enrollment_id,
65     mr.student_id,
66     mr.course_id,
67     mr.instructor_id,
68     mr.department_id,
69     mr.credits,
70     1,
71     mr.enrollment_date
72 FROM matched_rows mr
73 ON CONFLICT (source_enrollment_id) DO UPDATE
74 SET student_id = EXCLUDED.student_id,
75     course_id = EXCLUDED.course_id,
76     instructor_id = EXCLUDED.instructor_id,
77     department_id = EXCLUDED.department_id,
78     credits = EXCLUDED.credits,
79     enrollment_count = EXCLUDED.enrollment_count,
80     enrollment_date = EXCLUDED.enrollment_date;
81
82 SELECT
83     fe.source_enrollment_id,
84     ds.student_name,
85     dc.course_name,
86     di.instructor_name,
87     dd.department_name,
88     fe.credits,
89     fe.enrollment_date
90 FROM public.fact_enrollment fe
91 JOIN public.dim_student ds
92     ON ds.student_id = fe.student_id
93 JOIN public.dim_course dc
94     ON dc.course_id = fe.course_id
95 JOIN public.dim_instructor di
96     ON di.instructor_id = fe.instructor_id
97 JOIN public.dim_department dd
98     ON dd.department_id = fe.department_id
99 WHERE fe.source_enrollment_id >= 6001
100 ORDER BY fe.source_enrollment_id;

```

Output:

|    | source_enrollment_id<br>integer | student_name<br>character varying (100) | course_name<br>character varying (100) | instructor_name<br>character varying (100) | department_name<br>character varying (100) | credits<br>integer | enrollment_date<br>date |
|----|---------------------------------|-----------------------------------------|----------------------------------------|--------------------------------------------|--------------------------------------------|--------------------|-------------------------|
| 1  | 6001                            | Ayesha Khan                             | Machine Learning                       | Teacher 4                                  | Data Science                               | 4                  | 2026-02-10              |
| 2  | 6002                            | Khalid Hussain                          | Cloud Computing                        | Teacher 3                                  | Data Science                               | 4                  | 2026-02-11              |
| 3  | 6003                            | Sara Noor                               | Computer Networks                      | Teacher 2                                  | Computer Science                           | 3                  | 2026-02-12              |
| 4  | 6004                            | Sheikh Khalid                           | Cyber Security                         | Teacher 5                                  | Electrical Engineering                     | 4                  | 2026-02-13              |
| 5  | 6005                            | Zainab Iqbal                            | Digital Logic Design                   | Teacher 5                                  | Electrical Engineering                     | 3                  | 2026-02-14              |
| 6  | 6006                            | Umer Farooq                             | Data Warehousing                       | Teacher 3                                  | Data Science                               | 4                  | 2026-02-15              |
| 7  | 6007                            | Fatima Zahra                            | Database Systems                       | Miss Arifa                                 | Computer Science                           | 3                  | 2026-02-16              |
| 8  | 6008                            | Muhammad Saad                           | Big Data Analytics                     | Teacher 2                                  | Computer Science                           | 3                  | 2026-02-17              |
| 9  | 6009                            | Noor Ul Ain                             | Machine Learning                       | Teacher 4                                  | Data Science                               | 4                  | 2026-02-18              |
| 10 | 6010                            | Ali Raza                                | Cloud Computing                        | Teacher 3                                  | Data Science                               | 4                  | 2026-02-19              |
| 11 | 6011                            | Hira Faisal                             | Cyber Security                         | Teacher 5                                  | Electrical Engineering                     | 4                  | 2026-02-20              |

06\_task02\_validation\_queries:

```

1  -- Task 02
2  -- 4. Add a fake course row.
3  -- 5. Write a LEFT JOIN query between stg_enrollments and Courses.
4  -- 6. Identify records that do not have a match in the warehouse.
5
6  INSERT INTO staging.stg_enrollments (
7      student_name_raw,
8      course_name_raw,
9      instructor_name_raw,
10     enrollment_date_raw
11 ) VALUES (
12     'Future Student ',
13     'Quantum Physics',
14     'Teacher 3',
15     '02/21/2026'
16 );
17
18 WITH cleaned_staging AS (
19     SELECT
20         se.stg_enrollment_id,
21         staging.normalize_name(se.student_name_raw) AS student_name,
22         staging.normalize_name(se.course_name_raw) AS course_name,
23         staging.normalize_name(se.instructor_name_raw) AS instructor_name,
24         TO_DATE(BTRIM(se.enrollment_date_raw), 'MM/DD/YYYY') AS enrollment_date
25     FROM staging.stg_enrollments se
26 )
27 SELECT
28     cs.stg_enrollment_id,
29     cs.student_name,
30     cs.course_name AS staging_course_name,
31     dc.course_id AS matched_course_id,
32     dc.course_name AS warehouse_course_name
33 FROM cleaned_staging cs
34 LEFT JOIN public.dim_course dc
35     ON staging.normalize_name(dc.course_name) = cs.course_name
36 ORDER BY cs.stg_enrollment_id;
37
38 WITH cleaned_staging AS (
39     SELECT
40         se.stg_enrollment_id,
41         staging.normalize_name(se.student_name_raw) AS student_name,
42         staging.normalize_name(se.course_name_raw) AS course_name,
43         staging.normalize_name(se.instructor_name_raw) AS instructor_name,
44         TO_DATE(BTRIM(se.enrollment_date_raw), 'MM/DD/YYYY') AS enrollment_date
45     FROM staging.stg_enrollments se
46 ),
47
48 warehouse_match_check AS (
49     SELECT
50         cs.stg_enrollment_id,
51         cs.student_name,
52         cs.course_name,
53         cs.instructor_name,
54         cs.enrollment_date,
55         dc.course_id,
56         di.instructor_id
57     FROM cleaned_staging cs

```

```

57     LEFT JOIN public.dim_course dc
58         ON staging.normalize_name(dc.course_name) = cs.course_name
59     LEFT JOIN public.dim_instructor di
60         ON staging.normalize_name(di.instructor_name) = cs.instructor_name
61 )
62 SELECT
63     stg_enrollment_id,
64     student_name,
65     course_name,
66     instructor_name,
67     enrollment_date
68 FROM warehouse_match_check
69 WHERE course_id IS NULL
70        OR instructor_id IS NULL
71 ORDER BY stg_enrollment_id;

```

Output:

|   | stg_enrollment_id<br>[PK] integer | student_name<br>text | course_name<br>text | instructor_name<br>text | enrollment_date<br>date |
|---|-----------------------------------|----------------------|---------------------|-------------------------|-------------------------|
| 1 | 12                                | Future Student       | Quantum Physi...    | Teacher 3               | 2026-02-21              |

07\_task03\_cleanup:

```

1  -- Task 03
2  -- 1. Empty stg_enrollments without deleting its structure.
3
4  TRUNCATE TABLE staging.stg_enrollments RESTART IDENTITY;
5
6  SELECT COUNT(*) AS remaining_rows
7  FROM staging.stg_enrollments;

```

Output:

|   | remaining_rows<br>bigint |
|---|--------------------------|
| 1 | 0                        |

## LAB 8

### 8.1. OBJECTIVE

**Setup** an automated ETL sequence by constructing SQL verification scripts

### 8.3. EXERCISES

For all tasks, use the Snipping Tool to take focused screenshots. Your screenshots should be clear and readable.

#### 8.3.1. Task 01

Work with the *university\_dw* created in Lab 03.

1. The Registrar sends a monthly CSV. You must automate the load into the Enrollments table. Write a SQL statement that automatically joins with the *Instructors* table to find the current *Instructor\_SK*. Ensure that only students with a valid *Course\_ID* are moved from staging to the Fact table.
2. Sometimes students are enrolled in courses that have been deleted from the Courses table. Write a query to identify any records in the Enrollments table that have a *Course\_ID* which no longer exists in the Courses dimension.
3. Create a Cleanup Script that moves the orphan records into a separate table called *error\_log\_enrollments*.
4. Using the *course\_fees.csv* data from Lab 06, write a script that updates the *Courses* table but only for records where the fee has changed. If the base fee in the staging area matches the current base fee in *Courses* table, do nothing. If it is different, update the record.
5. Automate the update of the Materialized View (*mv\_enrollment\_flat*) from Lab 03 as the final step of the ETL script. Why is refreshing the view the last step? What would happen if the view is refreshed before the *INSERT* command?

#### 01\_rebuild\_lab03\_foundation:

```

1  -- Rebuild the Lab 3 style foundation for Lab 8 automation work.
2
3  DROP MATERIALIZED VIEW IF EXISTS public.mv_enrollment_flat;
4  DROP TABLE IF EXISTS public.error_log_enrollments;
5  DROP TABLE IF EXISTS public.enrollments;
6  DROP TABLE IF EXISTS public.instructors;
7  DROP TABLE IF EXISTS public.courses;
8  DROP TABLE IF EXISTS public.departments;
9
10 DROP SCHEMA IF EXISTS staging CASCADE;
11 CREATE SCHEMA staging;
12
13 CREATE OR REPLACE FUNCTION staging.normalize_name(input_text TEXT)
14 RETURNS TEXT
15 LANGUAGE sql
16 IMMUTABLE
17 AS $$
18     SELECT NULLIF(
19         INITCAP(
20             BTRIM(
21                 REGEXP_REPLACE(COALESCE(input_text, ''), '\s+', ' ', 'g')
22             )
23         ),
24         ''
25     );
26 $$;
27
28 CREATE TABLE public.departments (
29     department_id INT PRIMARY KEY,
30     department_name VARCHAR(100) NOT NULL

```

```

31 );
32
33 CREATE TABLE public.courses (
34     course_id INT PRIMARY KEY,
35     course_name VARCHAR(100) NOT NULL,
36     credits INT NOT NULL CHECK (credits > 0),
37     base_fee NUMERIC(12,2)
38 );
39
40 CREATE TABLE public.instructors (
41     instructor_id INT PRIMARY KEY,
42     instructor_name VARCHAR(100) NOT NULL,
43     department_id INT NOT NULL REFERENCES public.departments(department_id)
44 );
45
46 CREATE TABLE public.enrollments (
47     enrollment_id INT PRIMARY KEY,
48     student_name VARCHAR(100) NOT NULL,
49     course_id INT NOT NULL,
50     instructor_id INT NOT NULL REFERENCES public.instructors(instructor_id),
51     enrollment_date DATE NOT NULL DEFAULT CURRENT_DATE,
52     load_source VARCHAR(30) NOT NULL DEFAULT 'seed'
53 );
54
55 CREATE TABLE public.error_log_enrollments (
56     error_log_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
57     source_system VARCHAR(50) NOT NULL,
58     source_enrollment_id INT,
59     student_name_raw VARCHAR(100),
60     course_id INT,
61     instructor_name_raw VARCHAR(100),
62     instructor_id INT,
63     enrollment_date_raw VARCHAR(20),
64     error_reason VARCHAR(200) NOT NULL,
65     logged_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
66     CONSTRAINT uq_error_log_enrollments
67         UNIQUE (source_system, source_enrollment_id, error_reason)
68 );
69
70 CREATE INDEX idx_enrollments_course_id ON public.enrollments(course_id);
71 CREATE INDEX idx_enrollments_instructor_id ON public.enrollments(instructor_id);
72
73 INSERT INTO public.departments (department_id, department_name) VALUES
74     (1, 'Computer Science'),
75     (2, 'Data Science'),
76     (3, 'Electrical Engineering');
77
78 INSERT INTO public.courses (course_id, course_name, credits, base_fee) VALUES
79     (101, 'Database Systems', 3, 12000.00),
80     (102, 'Data Warehousing', 4, 16000.00),
81     (103, 'Machine Learning', 4, 14000.00),
82     (104, 'Big Data Analytics', 3, 13500.00),
83     (105, 'Digital Logic Design', 3, 17000.00),
84     (106, 'Cloud Computing', 4, 24000.00),
85     (107, 'Computer Networks', 3, 10000.00),
86     (108, 'Cyber Security', 4, 16500.00);
87
88 INSERT INTO public.instructors (instructor_id, instructor_name, department_id) VALUES
89     (1, 'Miss Arifa', 1),
90     (2, 'Teacher 2', 1),
91     (3, 'Teacher 3', 2),
92     (4, 'Teacher 4', 2),
93     (5, 'Teacher 5', 3);
94
95 INSERT INTO public.enrollments (
96     enrollment_id,
97     student_name,
98     course_id,
99     instructor_id,
100     enrollment_date,
101     load_source
102 ) VALUES
103     (1, 'Sheikh Khalid', 101, 1, DATE '2026-02-01', 'seed'),
104     (2, 'Khalid Hussain', 101, 1, DATE '2026-02-02', 'seed'),
105     (3, 'Sheikh Khalid Hussain', 102, 3, DATE '2026-02-03', 'seed'),
106     (4, 'Student 4', 102, 3, DATE '2026-02-04', 'seed'),
107     (5, 'Student 5', 103, 4, DATE '2026-02-05', 'seed'),
108     (6, 'Student 6', 103, 4, DATE '2026-02-06', 'seed'),
109     (7, 'Student 7', 104, 2, DATE '2026-02-07', 'seed'),
110     (8, 'Student 8', 105, 5, DATE '2026-02-08', 'seed'),
111     (9, 'Student 9', 102, 3, DATE '2026-02-09', 'seed'),
112     (10, 'Student 10', 102, 3, DATE '2026-02-10', 'seed');
113
114 SELECT 'departments' AS table_name, COUNT(*) AS total_rows FROM public.departments
115 UNION ALL
116 SELECT 'courses', COUNT(*) FROM public.courses
117 UNION ALL
118 SELECT 'instructors', COUNT(*) FROM public.instructors
119 UNION ALL
120 SELECT 'enrollments', COUNT(*) FROM public.enrollments
121 ORDER BY table_name;

```

**Output:**

|   | table_name  | total_rows |
|---|-------------|------------|
|   | text        | bigint     |
| 1 | courses     | 8          |
| 2 | departments | 3          |
| 3 | enrollments | 10         |
| 4 | instructors | 5          |

**02\_materialized\_view\_setup:**

```

1  -- Create the reporting materialized view from Lab 3.
2
3  DROP MATERIALIZED VIEW IF EXISTS public.mv_enrollment_flat;
4
5  CREATE MATERIALIZED VIEW public.mv_enrollment_flat AS
6  SELECT
7      e.enrollment_id,
8      e.student_name,
9      e.enrollment_date,
10     e.load_source,
11     c.course_id,
12     c.course_name,
13     c.credits,
14     c.base_fee,
15     i.instructor_id,
16     i.instructor_name,
17     d.department_name
18 FROM public.enrollments e
19 JOIN public.courses c
20     ON c.course_id = e.course_id
21 JOIN public.instructors i
22     ON i.instructor_id = e.instructor_id
23 JOIN public.departments d
24     ON d.department_id = i.department_id;
25
26 SELECT
27     enrollment_id,
28     student_name,
29     course_name,
30     instructor_name,
31     department_name,
32     base_fee
33 FROM public.mv_enrollment_flat
34 ORDER BY enrollment_id;

```

**Output:**

|    | enrollment_id | student_name            | course_name             | instructor_name         | department_name         | base_fee       |
|----|---------------|-------------------------|-------------------------|-------------------------|-------------------------|----------------|
|    | integer       | character varying (100) | character varying (100) | character varying (100) | character varying (100) | numeric (12,2) |
| 1  | 1             | Sheikh Khalid           | Database Systems        | Miss Arifa              | Computer Science        | 12000.00       |
| 2  | 2             | Khalid Hussain          | Database Systems        | Miss Arifa              | Computer Science        | 12000.00       |
| 3  | 3             | Sheikh Khalid Hussain   | Data Warehousing        | Teacher 3               | Data Science            | 16000.00       |
| 4  | 4             | Student 4               | Data Warehousing        | Teacher 3               | Data Science            | 16000.00       |
| 5  | 5             | Student 5               | Machine Learning        | Teacher 4               | Data Science            | 14000.00       |
| 6  | 6             | Student 6               | Machine Learning        | Teacher 4               | Data Science            | 14000.00       |
| 7  | 7             | Student 7               | Big Data Analytics      | Teacher 2               | Computer Science        | 13500.00       |
| 8  | 8             | Student 8               | Digital Logic Design    | Teacher 5               | Electrical Engineering  | 17000.00       |
| 9  | 9             | Student 9               | Data Warehousing        | Teacher 3               | Data Science            | 16000.00       |
| 10 | 10            | Student 10              | Data Warehousing        | Teacher 3               | Data Science            | 16000.00       |

**03\_stage\_monthly\_enrollments:**

```

1  -- Load the monthly registrar CSV in a staging table.
2
3  DROP TABLE IF EXISTS staging.stg_monthly_enrollments;
4

```

```

5 CREATE TABLE staging.stg_monthly_enrollments (
6   source_enrollment_id INT PRIMARY KEY,
7   student_name_raw VARCHAR(100) NOT NULL,
8   course_id INT NOT NULL,
9   instructor_name_raw VARCHAR(100) NOT NULL,
10  enrollment_date_raw VARCHAR(20) NOT NULL
11 );
12
13 INSERT INTO staging.stg_monthly_enrollments (
14   source_enrollment_id,
15   student_name_raw,
16   course_id,
17   instructor_name_raw,
18   enrollment_date_raw
19 ) VALUES
20   (5001, ' Khalid Hussain ', 103, ' teacher 4 ', '03/01/2026'),
21   (5002, 'bilal ahmed', 102, 'TEACHER 3', '03/02/2026'),
22   (5003, 'SARA noor', 104, ' teacher 2', '03/03/2026'),
23   (5004, ' fatima zahra', 101, 'MISS ARIFA', '03/04/2026'),
24   (5005, 'Hira Faizal', 105, 'Teacher 5 ', '03/05/2026'),
25   (5006, 'Future Student', 999, 'Teacher 3', '03/06/2026'),
26   (5007, 'Noor ul ain', 103, 'teacher 4', '03/07/2026');
27
28 SELECT
29   source_enrollment_id,
30   student_name_raw,
31   course_id,
32   instructor_name_raw,
33   enrollment_date_raw
34 FROM staging.stg_monthly_enrollments
35 ORDER BY source_enrollment_id;

```

**Output:**

|   | source_enrollment_id<br>[PK] integer | student_name_raw<br>character varying (100) | course_id<br>integer | instructor_name_raw<br>character varying (100) | enrollment_date_raw<br>character varying (20) |
|---|--------------------------------------|---------------------------------------------|----------------------|------------------------------------------------|-----------------------------------------------|
| 1 | 5001                                 | Khalid Hussain                              | 103                  | teacher 4                                      | 03/01/2026                                    |
| 2 | 5002                                 | bilal ahmed                                 | 102                  | TEACHER 3                                      | 03/02/2026                                    |
| 3 | 5003                                 | SARA noor                                   | 104                  | teacher 2                                      | 03/03/2026                                    |
| 4 | 5004                                 | fatima zahra                                | 101                  | MISS ARIFA                                     | 03/04/2026                                    |
| 5 | 5005                                 | Hira Faizal                                 | 105                  | Teacher 5                                      | 03/05/2026                                    |
| 6 | 5006                                 | Future Student                              | 999                  | Teacher 3                                      | 03/06/2026                                    |
| 7 | 5007                                 | Noor ul ain                                 | 103                  | teacher 4                                      | 03/07/2026                                    |

**04 task01 monthly enrollment load:**

```

1  -- Task 01
2  -- Load only valid monthly enrollment rows and resolve the current instructor key.
3
4  CREATE TABLE IF NOT EXISTS public.error_log_enrollments (
5    error_log_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
6    source_system VARCHAR(50) NOT NULL,
7    source_enrollment_id INT,
8    student_name_raw VARCHAR(100),
9    course_id INT,
10   instructor_name_raw VARCHAR(100),
11   instructor_id INT,
12   enrollment_date_raw VARCHAR(20),
13   error_reason VARCHAR(200) NOT NULL,
14   logged_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
15   CONSTRAINT uq_error_log_enrollments
16     UNIQUE (source_system, source_enrollment_id, error_reason)
17 );
18
19 WITH cleaned_staging AS (
20   SELECT
21     sme.source_enrollment_id,
22     sme.student_name_raw,
23     staging.normalize_name(sme.student_name_raw) AS student_name,
24     sme.course_id,
25     sme.instructor_name_raw,
26     staging.normalize_name(sme.instructor_name_raw) AS instructor_name,
27     sme.enrollment_date_raw,
28     TO_DATE(BTRIM(sme.enrollment_date_raw), 'MM/DD/YYYY') AS enrollment_date
29   FROM staging.stg_monthly_enrollments sme
30 ),
31 validation AS (
32   SELECT
33     cs.*,
34     c.course_id AS matched_course_id,
35     i.instructor_id AS matched_instructor_id
36   FROM cleaned_staging cs
37   LEFT JOIN public.courses c
38     ON c.course_id = cs.course_id
39   LEFT JOIN public.instructors i
40     ON staging.normalize_name(i.instructor_name) = cs.instructor_name
41 ),
42 logged_rejects AS (
43   INSERT INTO public.error_log_enrollments (
44     source_system,
45     source_enrollment_id,
46     student_name_raw,

```

```

47     course_id,
48     instructor_name_raw,
49     instructor_id,
50     enrollment_date_raw,
51     error_reason
52 )
53 SELECT
54     'monthly_csv',
55     v.source_enrollment_id,
56     v.student_name_raw,
57     v.course_id,
58     v.instructor_name_raw,
59     v.matched_instructor_id,
60     v.enrollment_date_raw,
61     CASE
62         WHEN v.matched_course_id IS NULL THEN
63             'Rejected during load: course_id not found in public.courses'
64         WHEN v.matched_instructor_id IS NULL THEN
65             'Rejected during load: instructor_name not found in public.instructors'
66     END
67 FROM validation v
68 WHERE v.matched_course_id IS NULL
69 OR v.matched_instructor_id IS NULL
70 ON CONFLICT DO NOTHING
71 RETURNING source_enrollment_id
72 )
73 INSERT INTO public.enrollments (
74     enrollment_id,
75     student_name,
76     course_id,
77     instructor_id,
78     enrollment_date,
79     load_source
80 )
81 SELECT
82     v.source_enrollment_id,
83     v.student_name,
84     v.course_id,
85     v.matched_instructor_id,
86     v.enrollment_date,
87     'monthly_csv'
88 FROM validation v
89 WHERE v.matched_course_id IS NOT NULL
90 AND v.matched_instructor_id IS NOT NULL
91 ON CONFLICT (enrollment_id) DO UPDATE
92 SET student_name = EXCLUDED.student_name,

```

```

93     course_id = EXCLUDED.course_id,
94     instructor_id = EXCLUDED.instructor_id,
95     enrollment_date = EXCLUDED.enrollment_date,
96     load_source = EXCLUDED.load_source;
97
98 SELECT
99     e.enrollment_id,
100     e.student_name,
101     e.course_id,
102     e.course_name,
103     e.instructor_id,
104     i.instructor_name,
105     e.enrollment_date,
106     e.load_source
107 FROM public.enrollments e
108 JOIN public.courses c
109     ON c.course_id = e.course_id
110 JOIN public.instructors i
111     ON i.instructor_id = e.instructor_id
112 WHERE e.load_source = 'monthly_csv'
113 ORDER BY e.enrollment_id;
114
115 SELECT
116     source_enrollment_id,
117     student_name_raw,
118     course_id,
119     instructor_name_raw,
120     error_reason
121 FROM public.error_log_enrollments
122 ORDER BY error_log_id;

```

**Output:**

|   | source_enrollment_id<br>integer | student_name_raw<br>character varying (100) | course_id<br>integer | instructor_name_raw<br>character varying (100) | error_reason<br>character varying (200)                     |
|---|---------------------------------|---------------------------------------------|----------------------|------------------------------------------------|-------------------------------------------------------------|
| 1 | 5006                            | Future Student                              | 999                  | Teacher 3                                      | Rejected during load: course_id not found in public.cour... |

**05\_task02\_identify\_orphan\_enrollments:**

```

1  -- Task 02
2  -- Find enrollment rows whose course_id no longer exists in the Courses table.
3
4  SELECT
5      e.enrollment_id,
6      e.student_name,
7      e.course_id,
8      e.instructor_id,
9      e.enrollment_date,
10     e.load_source
11 FROM public.enrollments e
12 LEFT JOIN public.courses c
13     ON c.course_id = e.course_id
14 WHERE c.course_id IS NULL
15 ORDER BY e.enrollment_id;

```

**Output:**

| enrollment_id | student_name            | course_id | instructor_id | enrollment_date | load_source            |
|---------------|-------------------------|-----------|---------------|-----------------|------------------------|
| [PK] integer  | character varying (100) | integer   | integer       | date            | character varying (30) |

**06\_task03\_cleanup\_orphan\_enrollments:**

```

1  -- Task 03
2  -- Move orphan enrollments to the error log and remove them from the live table.
3
4  CREATE TABLE IF NOT EXISTS public.error_log_enrollments (
5      error_log_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
6      source_system VARCHAR(50) NOT NULL,
7      source_enrollment_id INT,
8      student_name_raw VARCHAR(100),
9      course_id INT,
10     instructor_name_raw VARCHAR(100),
11     instructor_id INT,
12     enrollment_date_raw VARCHAR(20),
13     error_reason VARCHAR(200) NOT NULL,
14     logged_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
15     CONSTRAINT uq_error_log_enrollments
16         UNIQUE (source_system, source_enrollment_id, error_reason)
17 );
18
19 WITH orphan_rows AS (
20     SELECT
21         e.enrollment_id,
22         e.student_name,
23         e.course_id,
24         e.instructor_id,
25         e.enrollment_date,
26         e.load_source
27     FROM public.enrollments e
28     LEFT JOIN public.courses c
29         ON c.course_id = e.course_id
30     WHERE c.course_id IS NULL
31 ),
32 logged_rows AS (
33     INSERT INTO public.error_log_enrollments (
34         source_system,
35         source_enrollment_id,
36         student_name_raw,
37         course_id,
38         instructor_name_raw,
39         instructor_id,
40         enrollment_date_raw,
41         error_reason
42     )
43     SELECT
44         COALESCE(o.load_source, 'public.enrollments'),
45         o.enrollment_id,
46         o.student_name,

```

```

47     o.course_id,
48     i.instructor_name,
49     o.instructor_id,
50     TO_CHAR(o.enrollment_date, 'YYYY-MM-DD'),
51     'Moved from public.enrollments because course_id no longer exists in public.courses'
52 FROM orphan_rows o
53 LEFT JOIN public.instructors i
54     ON i.instructor_id = o.instructor_id
55 RETURNING source_enrollment_id
56 );
57 deleted_rows AS (
58     DELETE FROM public.enrollments e
59     USING orphan_rows o
60     WHERE e.enrollment_id = o.enrollment_id
61     RETURNING e.enrollment_id
62 )
63 SELECT
64     (SELECT COUNT(*) FROM logged_rows) AS logged_rows,
65     (SELECT COUNT(*) FROM deleted_rows) AS deleted_rows;

```

Output:

|   | logged_rows<br>bigint | deleted_rows<br>bigint |
|---|-----------------------|------------------------|
| 1 | 0                     | 0                      |

### 07 task04 sync course fees:

```

1  -- Task 04
2  -- Update course fees only where the staged fee differs from the current fee.
3
4  DROP TABLE IF EXISTS staging.stg_course_fees;
5
6  CREATE TABLE staging.stg_course_fees (
7      course_name VARCHAR(100) NOT NULL,
8      base_fee_text VARCHAR(20) NOT NULL
9  );
10
11 INSERT INTO staging.stg_course_fees (course_name, base_fee_text) VALUES
12     ('Machine Learning', '15,800'),
13     ('Cloud Computing', '25,000'),
14     ('Computer Networks', '10,000'),
15     ('Cyber Security', '17,000'),
16     ('Digital Logic Design', '18,000');
17
18 WITH cleaned_fees AS (
19     SELECT
20         staging.normalize_name(course_name) AS course_name,
21         REPLACE(base_fee_text, ',', '')::NUMERIC(12,2) AS staged_base_fee
22     FROM staging.stg_course_fees
23 ),
24 updated_rows AS (
25     UPDATE public.courses c
26     SET base_fee = cf.staged_base_fee
27     FROM cleaned_fees cf
28     WHERE staging.normalize_name(c.course_name) = cf.course_name
29     AND c.base_fee IS DISTINCT FROM cf.staged_base_fee
30     RETURNING c.course_id, c.course_name, c.base_fee
31 )
32 SELECT
33     course_id,
34     course_name,
35     base_fee AS updated_base_fee
36 FROM updated_rows
37 ORDER BY course_id;
38
39 SELECT
40     course_id,
41     course_name,
42     base_fee
43 FROM public.courses
44 WHERE course_name IN (
45     'Machine Learning',
46     'Cloud Computing',

```

```

47     'Computer Networks',
48     'Cyber Security',
49     'Digital Logic Design'
50 )
51 ORDER BY course_id;

```

**Output:**

|   | course_id<br>[PK] integer | course_name<br>character varying (100) | base_fee<br>numeric (12,2) |
|---|---------------------------|----------------------------------------|----------------------------|
| 1 | 103                       | Machine Learning                       | 15000.00                   |
| 2 | 105                       | Digital Logic Design                   | 18000.00                   |
| 3 | 106                       | Cloud Computing                        | 25000.00                   |
| 4 | 107                       | Computer Networks                      | 10000.00                   |
| 5 | 108                       | Cyber Security                         | 17000.00                   |

**08 task05 full etl and refresh:**

```

1  -- Task 05
2  -- Run the ETL end-to-end and refresh the materialized view as the last step.
3
4  CREATE TABLE IF NOT EXISTS public.error_log_enrollments (
5      error_log_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
6      source_system VARCHAR(50) NOT NULL,
7      source_enrollment_id INT,
8      student_name_raw VARCHAR(100),
9      course_id INT,
10     instructor_name_raw VARCHAR(100),
11     instructor_id INT,
12     enrollment_date_raw VARCHAR(20),
13     error_reason VARCHAR(200) NOT NULL,
14     logged_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
15     CONSTRAINT uq_error_log_enrollments
16         UNIQUE (source_system, source_enrollment_id, error_reason)
17 );
18
19 WITH cleaned_staging AS (
20     SELECT
21         sme.source_enrollment_id,
22         sme.student_name_raw,
23         staging.normalize_name(sme.student_name_raw) AS student_name,
24         sme.course_id,
25         sme.instructor_name_raw,
26         staging.normalize_name(sme.instructor_name_raw) AS instructor_name,
27         sme.enrollment_date_raw,
28         TO_DATE(BTRIM(sme.enrollment_date_raw), 'MM/DD/YYYY') AS enrollment_date
29     FROM staging.stg_monthly_enrollments sme
30 ),
31 validation AS (
32     SELECT
33         cs.*,
34         c.course_id AS matched_course_id,
35         i.instructor_id AS matched_instructor_id
36     FROM cleaned_staging cs
37     LEFT JOIN public.courses c
38         ON c.course_id = cs.course_id
39     LEFT JOIN public.instructors i
40         ON staging.normalize_name(i.instructor_name) = cs.instructor_name
41 )
42 INSERT INTO public.error_log_enrollments (
43     source_system,
44     source_enrollment_id,
45     student_name_raw,
46     course_id,
47     instructor_name_raw,
48     instructor_id,
49     enrollment_date_raw,
50     error_reason
51 )
52 SELECT
53     'monthly_csv',
54     v.source_enrollment_id,
55     v.student_name_raw,

```

```

56     v.course_id,
57     v.instructor_name_raw,
58     v.matched_instructor_id,
59     v.enrollment_date_raw,
60     CASE
61         WHEN v.matched_course_id IS NULL THEN
62             'Rejected during load: course_id not found in public.courses'
63         WHEN v.matched_instructor_id IS NULL THEN
64             'Rejected during load: instructor_name not found in public.instructors'
65     END
66 FROM validation v
67 WHERE v.matched_course_id IS NULL
68      OR v.matched_instructor_id IS NULL
69 ON CONFLICT DO NOTHING;
70
71 WITH cleaned_staging AS (
72     SELECT
73         sme.source_enrollment_id,
74         staging.normalize_name(sme.student_name_raw) AS student_name,
75         sme.course_id,
76         staging.normalize_name(sme.instructor_name_raw) AS instructor_name,
77         TO_DATE(BTRIM(sme.enrollment_date_raw), 'MM/DD/YYYY') AS enrollment_date
78     FROM staging.stg_monthly_enrollments sme
79 ),
80 valid_rows AS (
81     SELECT
82         cs.source_enrollment_id,
83         cs.student_name,
84         cs.course_id,
85         i.instructor_id,
86         cs.enrollment_date
87     FROM cleaned_staging cs
88     JOIN public.courses c
89         ON c.course_id = cs.course_id
90     JOIN public.instructors i
91         ON staging.normalize_name(i.instructor_name) = cs.instructor_name
92 )
93 INSERT INTO public.enrollments (
94     enrollment_id,
95     student_name,
96     course_id,
97     instructor_id,
98     enrollment_date,
99     load_source
100 )
101 SELECT

```

```

102     vr.source_enrollment_id,
103     vr.student_name,
104     vr.course_id,
105     vr.instructor_id,
106     vr.enrollment_date,
107     'monthly_csv'
108 FROM valid_rows vr
109 ON CONFLICT (enrollment_id) DO UPDATE
110 SET student_name = EXCLUDED.student_name,
111     course_id = EXCLUDED.course_id,
112     instructor_id = EXCLUDED.instructor_id,
113     enrollment_date = EXCLUDED.enrollment_date,
114     load_source = EXCLUDED.load_source;
115
116 WITH orphan_rows AS (
117     SELECT
118         e.enrollment_id,
119         e.student_name,
120         e.course_id,
121         e.instructor_id,
122         e.enrollment_date,
123         e.load_source
124     FROM public.enrollments e
125     LEFT JOIN public.courses c
126         ON c.course_id = e.course_id
127     WHERE c.course_id IS NULL
128 ),
129 logged_rows AS (
130     INSERT INTO public.error_log_enrollments (
131         source_system,
132         source_enrollment_id,
133         student_name_raw,
134         course_id,
135         instructor_name_raw,
136         instructor_id,
137         enrollment_date_raw,
138         error_reason
139     )
140     SELECT
141         COALESCE(o.load_source, 'public.enrollments'),
142         o.enrollment_id,
143         o.student_name,
144         o.course_id,
145         i.instructor_name,
146         o.instructor_id,
147         TO_CHAR(o.enrollment_date, 'YYYY-MM-DD'),

```

```

148     'Moved from public.enrollments because course_id no longer exists in public.courses'
149     FROM orphan_rows o
150     LEFT JOIN public.instructors i
151         ON i.instructor_id = o.instructor_id
152     RETURNING source_enrollment_id
153 )
154 DELETE FROM public.enrollments e
155 USING orphan_rows o
156 WHERE e.enrollment_id = o.enrollment_id;
157
158 DROP TABLE IF EXISTS staging.stg_course_fees;
159
160 CREATE TABLE staging.stg_course_fees (
161     course_name VARCHAR(100) NOT NULL,
162     base_fee_text VARCHAR(20) NOT NULL
163 );
164
165 INSERT INTO staging.stg_course_fees (course_name, base_fee_text) VALUES
166     ('Machine Learning', '15,000'),
167     ('Cloud Computing', '25,000'),
168     ('Computer Networks', '10,000'),
169     ('Cyber Security', '17,000'),
170     ('Digital Logic Design', '18,000');
171
172 WITH cleaned_fees AS (
173     SELECT
174         staging.normalize_name(course_name) AS course_name,
175         REPLACE(base_fee_text, ',', '')::NUMERIC(12,2) AS staged_base_fee
176     FROM staging.stg_course_fees
177 )
178 UPDATE public.courses c
179 SET base_fee = cf.staged_base_fee
180 FROM cleaned_fees cf
181 WHERE staging.normalize_name(c.course_name) = cf.course_name
182 AND c.base_fee IS DISTINCT FROM cf.staged_base_fee;
183
184 REFRESH MATERIALIZED VIEW public.mv_enrollment_flat;
185
186 SELECT
187     (SELECT COUNT(*) FROM staging.stg_monthly_enrollments) AS staged_rows,
188     (SELECT COUNT(*) FROM public.enrollments WHERE load_source = 'monthly_csv') AS monthly_rows_in_enrollments,
189     (SELECT COUNT(*) FROM public.error_log_enrollments) AS rejected_or_logged_rows,
190     (SELECT COUNT(*) FROM public.mv_enrollment_flat) AS rows_in_mv_enrollment_flat;
191
192 SELECT
193     enrollment_id,
194     student_name,
195     course_name,
196     instructor_name,
197     department_name,
198     base_fee
199 FROM public.mv_enrollment_flat
200 WHERE load_source = 'monthly_csv'
201 ORDER BY enrollment_id;

```

**Output:**

|   | enrollment_id<br>integer | student_name<br>character varying (100) | course_name<br>character varying (100) | instructor_name<br>character varying (100) | department_name<br>character varying (100) | base_fee<br>numeric (12,2) |
|---|--------------------------|-----------------------------------------|----------------------------------------|--------------------------------------------|--------------------------------------------|----------------------------|
| 1 | 5001                     | Khalid Hussain                          | Machine Learning                       | Teacher 4                                  | Data Science                               | 15000.00                   |
| 2 | 5002                     | Bilal Ahmed                             | Data Warehousing                       | Teacher 3                                  | Data Science                               | 16000.00                   |
| 3 | 5003                     | Sara Noor                               | Big Data Analytics                     | Teacher 2                                  | Computer Science                           | 13500.00                   |
| 4 | 5004                     | Fatima Zahra                            | Database Systems                       | Miss Arifa                                 | Computer Science                           | 12000.00                   |
| 5 | 5005                     | Hira Faisal                             | Digital Logic Design                   | Teacher 5                                  | Electrical Engineering                     | 18000.00                   |
| 6 | 5007                     | Noor Ul Ain                             | Machine Learning                       | Teacher 4                                  | Data Science                               | 15000.00                   |

## LAB 9

### 9.1. OBJECTIVE

**Construct** an OLAP Cube to execute Slice, Dice, and Drill-down operations

### 9.3. EXERCISES

For all tasks, use the Snipping Tool to take focused screenshots. Your screenshots should be clear and readable.

#### 9.3.1. Task 01

Work with the data warehouse for the *university\_dw* created in Lab 04.

1. Create a view named *v\_enrollment\_cube* that aggregates
  - a. Dimensions: Department Name, Course Name, Instructor Name, Semester/Year
  - b. Measures: Total Enrollments

#### Source Code:

```
1  -- LAB 9
2  -- Khalid Hussain (2022F-BCE-130)
3
4  -- Use the Lab 4 university_dw warehouse.
5  -- This lab demonstrates OLAP Cube operations:
6  -- Slice, Dice, and Drill-Down.
7
8  -----
9  -- Task 01
10 -- Add Semester and Year columns to support OLAP analysis.
11 -----
12
13 ALTER TABLE public.fact_enrollment
14 ADD COLUMN IF NOT EXISTS semester VARCHAR(20);
15
16 ALTER TABLE public.fact_enrollment
17 ADD COLUMN IF NOT EXISTS academic_year INT;
18
19 -----
20 -- Populate sample semester/year values.
21 -----
22
23 UPDATE public.fact_enrollment
24 SET
25     semester = CASE
26         WHEN source_enrollment_id BETWEEN 1 AND 5
27         THEN 'Spring'
28         ELSE 'Fall'
29     END,
30     academic_year = 2026;
31
32 -----
33 -- Create OLAP Cube View
34 -----
35
36 DROP VIEW IF EXISTS public.v_enrollment_cube;
37
```

```

38 CREATE VIEW public.v_enrollment_cube AS
39 SELECT
40     d.department_name,
41     c.course_name,
42     i.instructor_name,
43     f.semester,
44     f.academic_year,
45     SUM(f.enrollment_count) AS total_enrollments
46 FROM public.fact_enrollment f
47 JOIN public.dim_department d
48     ON d.department_id = f.department_id
49 JOIN public.dim_course c
50     ON c.course_id = f.course_id
51 JOIN public.dim_instructor i
52     ON i.instructor_id = f.instructor_id
53 GROUP BY CUBE (
54     d.department_name,
55     c.course_name,
56     i.instructor_name,
57     f.semester,
58     f.academic_year
59 );
60
61 -----
62 -- Display the OLAP Cube
63 -----
64
65 SELECT *
66 FROM public.v_enrollment_cube
67 ORDER BY
68     department_name,
69     course_name,
70     instructor_name;

```

**Output:**

|    | department_name<br>character varying (100) | course_name<br>character varying (100) | instructor_name<br>character varying (100) | semester<br>character varying (20) | academic_year<br>integer | total_enrollments<br>bigint |
|----|--------------------------------------------|----------------------------------------|--------------------------------------------|------------------------------------|--------------------------|-----------------------------|
| 1  | Computer Science                           | Big Data Analytics                     | Teacher 2                                  | Fall                               | [null]                   | 2                           |
| 2  | Computer Science                           | Big Data Analytics                     | Teacher 2                                  | [null]                             | 2026                     | 2                           |
| 3  | Computer Science                           | Big Data Analytics                     | Teacher 2                                  | [null]                             | [null]                   | 2                           |
| 4  | Computer Science                           | Big Data Analytics                     | Teacher 2                                  | Fall                               | 2026                     | 2                           |
| 5  | Computer Science                           | Big Data Analytics                     | [null]                                     | [null]                             | [null]                   | 2                           |
| 6  | Computer Science                           | Big Data Analytics                     | [null]                                     | [null]                             | 2026                     | 2                           |
| 7  | Computer Science                           | Big Data Analytics                     | [null]                                     | Fall                               | 2026                     | 2                           |
| 8  | Computer Science                           | Big Data Analytics                     | [null]                                     | Fall                               | [null]                   | 2                           |
| 9  | Computer Science                           | Computer Networks                      | Teacher 2                                  | [null]                             | [null]                   | 1                           |
| 10 | Computer Science                           | Computer Networks                      | Teacher 2                                  | Fall                               | [null]                   | 1                           |
| 11 | Computer Science                           | Computer Networks                      | Teacher 2                                  | [null]                             | 2026                     | 1                           |
| 12 | Computer Science                           | Computer Networks                      | Teacher 2                                  | Fall                               | 2026                     | 1                           |
| 13 | Computer Science                           | Computer Networks                      | [null]                                     | [null]                             | 2026                     | 1                           |
| 14 | Computer Science                           | Computer Networks                      | [null]                                     | [null]                             | [null]                   | 1                           |
| 15 | Computer Science                           | Computer Networks                      | [null]                                     | Fall                               | [null]                   | 1                           |
| 16 | Computer Science                           | Computer Networks                      | [null]                                     | Fall                               | 2026                     | 1                           |
| 17 | Computer Science                           | Database Systems                       | Miss Arifa                                 | [null]                             | [null]                   | 3                           |
| 18 | Computer Science                           | Database Systems                       | Miss Arifa                                 | Fall                               | 2026                     | 1                           |
| 19 | Computer Science                           | Database Systems                       | Miss Arifa                                 | Spring                             | [null]                   | 2                           |
| 20 | Computer Science                           | Database Systems                       | Miss Arifa                                 | Fall                               | [null]                   | 1                           |
| 21 | Computer Science                           | Database Systems                       | Miss Arifa                                 | [null]                             | 2026                     | 3                           |
| 22 | Computer Science                           | Database Systems                       | Miss Arifa                                 | Spring                             | 2026                     | 2                           |
| 23 | Computer Science                           | Database Systems                       | [null]                                     | Fall                               | [null]                   | 1                           |
| 24 | Computer Science                           | Database Systems                       | [null]                                     | [null]                             | [null]                   | 3                           |
| 25 | Computer Science                           | Database Systems                       | [null]                                     | Spring                             | [null]                   | 2                           |
| 26 | Computer Science                           | Database Systems                       | [null]                                     | [null]                             | 2026                     | 3                           |

2. Write a query to show total enrollments for only the one specific department, e.g., Computer Engineering.

**Source Code:**

```

1  -- Slice Operation
2  -- Example: Computer Science Department
3
4  SELECT
5      department_name,
6      course_name,
7      instructor_name,
8      semester,
9      academic_year,
10     total_enrollments
11 FROM public.v_enrollment_cube
12 WHERE department_name = 'Computer Science'
13 ORDER BY
14     course_name,
15     instructor_name;

```

**Output:**

|                                               | department_name<br>character varying (100) | course_name<br>character varying (100) | instructor_name<br>character varying (100) | semester<br>character varying (20) | academic_year<br>integer | total_enrollments<br>bigint |
|-----------------------------------------------|--------------------------------------------|----------------------------------------|--------------------------------------------|------------------------------------|--------------------------|-----------------------------|
| 1                                             | Computer Science                           | Big Data Analytics                     | Teacher 2                                  | [null]                             | 2026                     | 2                           |
| 2                                             | Computer Science                           | Big Data Analytics                     | Teacher 2                                  | [null]                             | [null]                   | 2                           |
| 3                                             | Computer Science                           | Big Data Analytics                     | Teacher 2                                  | Fall                               | 2026                     | 2                           |
| 4                                             | Computer Science                           | Big Data Analytics                     | Teacher 2                                  | Fall                               | [null]                   | 2                           |
| 5                                             | Computer Science                           | Big Data Analytics                     | [null]                                     | Fall                               | 2026                     | 2                           |
| 6                                             | Computer Science                           | Big Data Analytics                     | [null]                                     | Fall                               | [null]                   | 2                           |
| 7                                             | Computer Science                           | Big Data Analytics                     | [null]                                     | [null]                             | 2026                     | 2                           |
| 8                                             | Computer Science                           | Big Data Analytics                     | [null]                                     | [null]                             | [null]                   | 2                           |
| 9                                             | Computer Science                           | Computer Networks                      | Teacher 2                                  | [null]                             | 2026                     | 1                           |
| 10                                            | Computer Science                           | Computer Networks                      | Teacher 2                                  | [null]                             | [null]                   | 1                           |
| 11                                            | Computer Science                           | Computer Networks                      | Teacher 2                                  | Fall                               | 2026                     | 1                           |
| 12                                            | Computer Science                           | Computer Networks                      | Teacher 2                                  | Fall                               | [null]                   | 1                           |
| 13                                            | Computer Science                           | Computer Networks                      | [null]                                     | Fall                               | [null]                   | 1                           |
| 14                                            | Computer Science                           | Computer Networks                      | [null]                                     | Fall                               | 2026                     | 1                           |
| 15                                            | Computer Science                           | Computer Networks                      | [null]                                     | [null]                             | 2026                     | 1                           |
| 16                                            | Computer Science                           | Computer Networks                      | [null]                                     | [null]                             | [null]                   | 1                           |
| 17                                            | Computer Science                           | Database Systems                       | Miss Arifa                                 | [null]                             | [null]                   | 3                           |
| 18                                            | Computer Science                           | Database Systems                       | Miss Arifa                                 | Fall                               | [null]                   | 1                           |
| 19                                            | Computer Science                           | Database Systems                       | Miss Arifa                                 | Spring                             | [null]                   | 2                           |
| 20                                            | Computer Science                           | Database Systems                       | Miss Arifa                                 | Spring                             | 2026                     | 2                           |
| 21                                            | Computer Science                           | Database Systems                       | Miss Arifa                                 | Fall                               | 2026                     | 1                           |
| 22                                            | Computer Science                           | Database Systems                       | Miss Arifa                                 | [null]                             | 2026                     | 3                           |
| 23                                            | Computer Science                           | Database Systems                       | [null]                                     | Spring                             | 2026                     | 2                           |
| 24                                            | Computer Science                           | Database Systems                       | [null]                                     | [null]                             | 2026                     | 3                           |
| 25                                            | Computer Science                           | Database Systems                       | [null]                                     | Spring                             | [null]                   | 2                           |
| 26                                            | Computer Science                           | Database Systems                       | [null]                                     | [null]                             | [null]                   | 3                           |
| Total rows: 44    Query complete 00:00:00.278 |                                            |                                        |                                            |                                    |                          |                             |

|    |                  |                  |            |        |        |   |
|----|------------------|------------------|------------|--------|--------|---|
| 27 | Computer Science | Database Systems | [null]     | Fall   | 2026   | 1 |
| 28 | Computer Science | Database Systems | [null]     | Fall   | [null] | 1 |
| 29 | Computer Science | [null]           | Miss Arifa | Fall   | [null] | 1 |
| 30 | Computer Science | [null]           | Miss Arifa | [null] | 2026   | 3 |
| 31 | Computer Science | [null]           | Miss Arifa | Spring | [null] | 2 |
| 32 | Computer Science | [null]           | Miss Arifa | [null] | [null] | 3 |
| 33 | Computer Science | [null]           | Miss Arifa | Spring | 2026   | 2 |
| 34 | Computer Science | [null]           | Miss Arifa | Fall   | 2026   | 1 |
| 35 | Computer Science | [null]           | Teacher 2  | [null] | 2026   | 3 |
| 36 | Computer Science | [null]           | Teacher 2  | [null] | [null] | 3 |
| 37 | Computer Science | [null]           | Teacher 2  | Fall   | 2026   | 3 |
| 38 | Computer Science | [null]           | Teacher 2  | Fall   | [null] | 3 |
| 39 | Computer Science | [null]           | [null]     | Spring | 2026   | 2 |
| 40 | Computer Science | [null]           | [null]     | Fall   | [null] | 4 |
| 41 | Computer Science | [null]           | [null]     | Fall   | 2026   | 4 |
| 42 | Computer Science | [null]           | [null]     | [null] | [null] | 6 |
| 43 | Computer Science | [null]           | [null]     | Spring | [null] | 2 |
| 44 | Computer Science | [null]           | [null]     | [null] | 2026   | 6 |

3. Write a query to show how many students were enrolled in a specific course taught by a particular instructor in a specific semester, e.g., Data Warehousing by Dr. John Smith in Spring 2026.

#### Source Code:

```

1  -- Dice Operation
2  -- Example:
3  -- Data Warehousing taught by Teacher 3 in Spring 2026
4
5  SELECT
6      department_name,
7      course_name,
8      instructor_name,
9      semester,
10     academic_year,
11     total_enrollments
12 FROM public.v_enrollment_cube
13 WHERE course_name = 'Data Warehousing'
14        AND instructor_name = 'Teacher 3'
15        AND semester = 'Spring'
16        AND academic_year = 2026;
```

#### Output:

|   | department_name<br>character varying (100) | course_name<br>character varying (100) | instructor_name<br>character varying (100) | semester<br>character varying (20) | academic_year<br>integer | total_enrollments<br>bigint |
|---|--------------------------------------------|----------------------------------------|--------------------------------------------|------------------------------------|--------------------------|-----------------------------|
| 1 | [null]                                     | Data Warehousing                       | Teacher 3                                  | Spring                             | 2026                     | 2                           |
| 2 | Data Science                               | Data Warehousing                       | Teacher 3                                  | Spring                             | 2026                     | 2                           |

4. Generate a report that shows:
- Total students per Department.
  - Total students per Course within those departments.
  - The Grand Total of all students in the University.

#### Source Code:

```

1  -- Drill-Down Report
2
3  SELECT
4      d.department_name,
5      c.course_name,
6      SUM(f.enrollment_count) AS total_students
7  FROM public.fact_enrollment f
8  JOIN public.dim_department d
9      ON d.department_id = f.department_id
10 JOIN public.dim_course c
11     ON c.course_id = f.course_id
12 GROUP BY ROLLUP (
13     d.department_name,
14     c.course_name
15 )
16 ORDER BY
17     d.department_name,
18     c.course_name;

```

**Output:**

|    | department_name<br>character varying (100) 🔒 | course_name<br>character varying (100) 🔒 | total_students<br>bigint 🔒 |
|----|----------------------------------------------|------------------------------------------|----------------------------|
| 1  | Computer Science                             | Big Data Analytics                       | 2                          |
| 2  | Computer Science                             | Computer Networks                        | 1                          |
| 3  | Computer Science                             | Database Systems                         | 3                          |
| 4  | Computer Science                             | [null]                                   | 6                          |
| 5  | Data Science                                 | Cloud Computing                          | 2                          |
| 6  | Data Science                                 | Data Warehousing                         | 5                          |
| 7  | Data Science                                 | Machine Learning                         | 4                          |
| 8  | Data Science                                 | [null]                                   | 11                         |
| 9  | Electrical Engineering                       | Cyber Security                           | 2                          |
| 10 | Electrical Engineering                       | Digital Logic Design                     | 2                          |
| 11 | Electrical Engineering                       | [null]                                   | 4                          |
| 12 | [null]                                       | [null]                                   | 21                         |